

Ingegneria del software

Appunti ragionati per la preparazione all'esame

Indice

1	Modelli di processo di sviluppo software	5
1.1	Il processo di sviluppo	5
1.2	Modelli di processo	6
1.3	Prospettiva storica	7
1.4	Tipi di modelli di processo	8
1.4.1	Code & Fix	8
1.4.2	Modello a cascata (Waterfall)	8
1.4.2.1	Le indicazioni di Royce	8
1.4.2.2	Studio di fattibilità	9
1.4.2.3	Varianti	9
1.4.3	V-Model	10
1.4.4	Modelli evolutivi	10
1.4.4.1	Prototipazione	10
1.4.4.2	Sviluppo iterativo-incrementale	11
1.4.5	Modello a spirale	11
1.4.6	Unified Process (UP/RUP)	12
1.4.7	Component-Based Software Engineering (CBSE)	13
1.4.8	Modello trasformatore e metodi formali	13
1.5	Approcci plan-driven e metodi agili	14
1.5.1	Pratiche di Extreme Programming	14
1.5.2	Limiti dei metodi agili	14
1.6	Come scegliere il modello	15
1.7	Riepilogo	16
2	Ingegneria dei requisiti	17
2.0.1	Perché i requisiti sono decisivi	17
2.0.2	Costo della scoperta tardiva	18
2.1	Livelli di dettaglio e rappresentazione	18
2.1.1	Requisiti utente	18
2.1.2	Requisiti di sistema	18
2.1.3	Modi di rappresentazione	19
2.2	Requisiti funzionali e non funzionali	20
2.2.1	Requisiti funzionali	20
2.2.2	Requisiti non funzionali	20
2.3	Qualità e verifica dei requisiti	21
2.3.1	Checklist essenziale	21
2.3.2	Dalla specifica al test	21
3	Use case	22
3.0.1	Utilità nel ciclo di sviluppo	22
3.1	Elementi di un modello di use case	22
3.1.1	Confine del sistema	23
3.1.2	Attori e ruoli	23
3.1.3	Associazioni	23
3.2	Scenari e descrizione testuale	24

3.2.1	Template di uno use case	24
3.2.2	Esempio: prelevare contante	25
3.2.3	Regole per i passi	25
3.3	Deviazioni, ripetizioni e scenari alternativi	26
3.3.1	Deviazioni semplici e complesse	26
3.3.2	Ripetizioni	26
3.3.3	Template di uno scenario alternativo	26
3.4	Relazioni nel diagramma UML	27
3.4.1	Inclusione (include)	27
3.4.2	Estensione (extend)	27
3.4.3	Generalizzazione di attori	28
3.4.4	Generalizzazione di use case	28
3.5	Mockup, qualità della scrittura e limiti	29
3.5.1	Screen mockup	29
3.5.2	Regole di buona scrittura	29
3.5.3	Punti di forza e limiti	29
3.5.4	Riepilogo	29
4	Design architetturale	30
4.0.1	Creatività e riuso	30
4.1	Architettura software e attributi di qualità	30
4.1.1	Perché esplicitare l'architettura	30
4.1.2	Componenti e connettori	30
4.1.3	Impatto sugli attributi di qualità	31
4.2	Stili architetturali strutturali	32
4.2.1	Layered architecture	32
4.2.2	Repository	32
4.2.3	Client-server	33
4.2.3.1	Two-tier e three-tier	33
4.2.4	Peer-to-peer (P2P)	33
4.2.5	Pipe and filter	34
4.3	Stili architetturali di controllo	35
4.3.1	Call-return	35
4.3.2	Manager model	35
4.3.3	Broadcast / publish-subscribe	36
4.3.3.1	Decisioni progettuali negli eventi	36
4.4	Architetture eterogenee	37
4.4.1	Esempio: compilatore	37
4.5	Microservices	38
4.5.1	Proprietà	38
4.5.2	API Gateway e REST	38
4.6	Riepilogo e scelta dello stile	39
5	Design delle componenti (Low-Level Design)	40
5.0.1	Fasi del processo di design	40
5.1	Design by Contract	40
5.1.1	Precondizione, postcondizione e invariante	41
5.1.2	Esempio: conto corrente	41
5.1.3	Responsabilità del cliente e del fornitore	41
5.1.4	Vantaggi	42
5.2	Progettazione degli algoritmi	43
5.2.1	Notazioni e PDL	43
5.2.2	Stepwise refinement	43
5.3	Principi di progettazione	44

5.3.1	Astrazione	44
5.3.2	Decomposizione	44
5.3.3	Modularità e Separation of Concerns	44
5.3.4	Information hiding	45
5.4	Riepilogo	46
6	Introduzione a UML	47
6.1	Modi d'uso e prospettive	47
6.1.1	Sketch, blueprint e linguaggio eseguibile	47
6.1.2	Prospettiva concettuale e software	47
6.1.3	Modello di dominio	48
6.2	Notazione, meta-modello e viste	49
6.2.1	Viste e modello 4+1	49
6.3	I diagrammi di UML 2.5	50
6.3.1	Diagrammi principali nel corso	50
6.4	UML nel processo di sviluppo	51
6.5	Profili, codice e uso pragmatico	52
6.5.1	Profili UML	52
6.5.2	UML e codice	52
6.5.3	Consigli pratici	52
6.6	Riepilogo	53
7	Class diagram e Object diagram UML	54
7.1	Classi, attributi e operazioni	54
7.1.1	Notazione della classe	54
7.1.2	Attributi	54
7.1.3	Molteplicità	54
7.1.4	Operazioni	55
7.1.5	Datatype, costruttori e membri statici	55
7.2	Associazioni e molteplicità	56
7.2.1	Attributo o associazione?	56
7.2.2	Associazioni riflessive	56
7.2.3	Associazioni bidirezionali	56
7.3	Object diagram e classi associative	57
7.3.1	Object diagram	57
7.3.2	Classe associativa	57
7.4	Relazioni tutto-parte e generalizzazione	58
7.4.1	Aggregazione e composizione	58
7.4.2	Generalizzazione ed ereditarietà	58
7.5	Dipendenze e traduzione nel codice	59
7.5.1	Dalle molteplicità alle strutture dati	59
7.6	Riepilogo	60
8	Interfacce e vincoli UML	61
8.1	Interfacce fornite e richieste	61
8.1.1	Realizzazione di un'interfaccia	61
8.1.2	Interfaccia richiesta	61
8.2	Due notazioni equivalenti	62
8.3	Perché usare le interfacce	63
8.4	Vincoli nei Class Diagram	64
8.4.1	Object Constraint Language	64
9	State Machine Diagram	65
9.1	Stato e comportamento	65
9.1.1	Modellazione statica e dinamica	65
9.2	Notazione e semantica	66

9.2.1 Stati, transizioni e pseudo-stati	66
9.2.2 Eventi	66
9.2.3 Run-to-completion	66
9.3 Esempio: ciclo di vita di una copia di libro	67
9.4 Attività interne	68
9.4.1 Entry, exit, do e reazioni interne	68
9.5 Stati composti e concorrenza	69
9.5.1 Stati composti	69
9.5.2 Regioni ortogonali	69
9.6 Quando usare una State Machine	70
9.6.1 Esempio: controllore di un forno	70
9.6.2 Esempio: workflow documentale	70
9.7 Implementazione	71
9.7.1 Switch sullo stato corrente	71
9.7.2 Design Pattern State	71
9.7.3 Tabelle di stato e generazione	71

1 Modelli di processo di sviluppo software

1.1 Il processo di sviluppo

Idea fondamentale

L'ingegneria del software tratta il software come un prodotto industriale: non considera soltanto il codice finale, ma soprattutto il **processo** con cui esso viene concepito, costruito, verificato e mantenuto.

Un **processo di sviluppo software** è un insieme strutturato e organizzato di attività finalizzate alla produzione di un sistema software. Stabilisce:

- **quali attività** devono essere svolte;
- **in quale ordine** devono avvenire;
- quali **regole** e responsabilità governano il lavoro;
- quali risultati intermedi, detti **artefatti**, devono essere prodotti.

Modellare il processo dà ordine, controllo e ripetibilità a un'attività altrimenti difficile da pianificare. Secondo Barry Boehm, un modello determina l'ordine delle fasi, i criteri per passare da una fase alla successiva e aiuta a rispondere a due domande operative: **che cosa fare dopo?** e **per quanto tempo?**

Processo e prodotto

Un processo migliore tende ad aumentare produttività e qualità, ma la relazione non è automatica: un buon processo riduce il rischio di errori sistematici, mentre la qualità finale dipende anche da persone, tecnologia, requisiti e contesto.

1.2 Modelli di processo

Un **modello di processo** è una rappresentazione astratta del processo reale. Non descrive ogni dettaglio operativo: seleziona le attività e le relazioni più importanti per guidare e coordinare il team.

I modelli si collocano lungo uno spettro tra due estremi:

Modello	Regole/pratiche	Grado di prescrittività
RUP	oltre 120	Molto prescrittivo e fortemente documentato
XP	13	Adattivo, ma con pratiche ingegneristiche precise
Scrum	9	Prescrive ruoli, eventi e iterazioni, non le tecniche di sviluppo
Kanban	3	Poche regole e flusso continuo
“Do whatever”	0	Nessuna struttura esplicita

Scrum è meno prescrittivo di XP perché non impone specifiche pratiche ingegneristiche; è però più prescrittivo di Kanban, poiché definisce iterazioni, ruoli ed eventi.

Attenzione alla terminologia

Non esiste una nomenclatura universale: nomi diversi possono indicare lo stesso modello e lo stesso nome può assumere significati diversi secondo autore e contesto. Bisogna verificare sempre le pratiche concrete associate al termine.

La scelta del modello spetta tipicamente a manager e project manager e dipende dal prodotto, dai rischi, dalle competenze, dalla dimensione del team e dai vincoli organizzativi. Non esiste un modello migliore in assoluto.

1.3 Prospettiva storica

Prima del 1965	Produzione artigianale e approccio Code & Fix.
1955–1960	Prime applicazioni pratiche di processi sequenziali in grandi progetti.
1965–1985	“Crisi del software”: costi, ritardi e qualità mostrano i limiti dello sviluppo non disciplinato.
1968	La conferenza NATO consacra il termine <i>software engineering</i> .
1970	Winston Royce descrive formalmente il processo sequenziale poi chiamato Waterfall.
Dal 1975	Si diffondono prototipazione e modelli evolutivi.
1986–1988	V-Model e modello a spirale di Barry Boehm.
Anni Novanta	Web engineering, CBSE, Unified Process ed Extreme Programming.
Dal 2000	Metodi agili, Model-Driven Development e, dal 2009 circa, DevOps.

Fred Brooks, nel saggio *No Silver Bullet*, sostiene che non esiste una singola innovazione capace di eliminare rapidamente la complessità essenziale dello sviluppo software.

1.4 Tipi di modelli di processo

I modelli seguenti rappresentano le principali strategie con cui organizzare lo sviluppo, dalla produzione non strutturata fino ai processi iterativi, guidati dal rischio, basati sul riuso o sui metodi formali.

1.4.1 Code & Fix

Code & Fix è l'approccio più elementare: si comprende il bisogno in modo informale, si scrive il codice, lo si esegue e si correggono gli errori fino alla consegna. Non esistono vere fasi di analisi e progettazione.

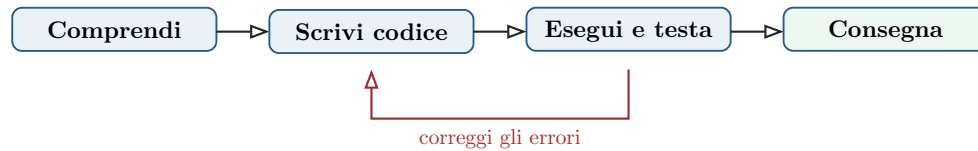


Figura 1: Flusso essenziale del Code & Fix.

È accettabile soltanto per programmi molto piccoli, indicativamente sotto le 1500 righe di codice. Su progetti grandi o multi-sviluppatore, l'assenza di piano, documentazione e controlli rende difficile coordinare il lavoro e governare la qualità.

1.4.2 Modello a cascata (Waterfall)

Il modello a cascata è il primo modello storico strutturato. Fu impiegato in grandi progetti, anche nel settore della difesa statunitense, e formalizzato da Winston Royce nel 1970. Introduce una sequenza di fasi: l'artefatto prodotto da ciascuna fase diventa l'ingresso di quella successiva.

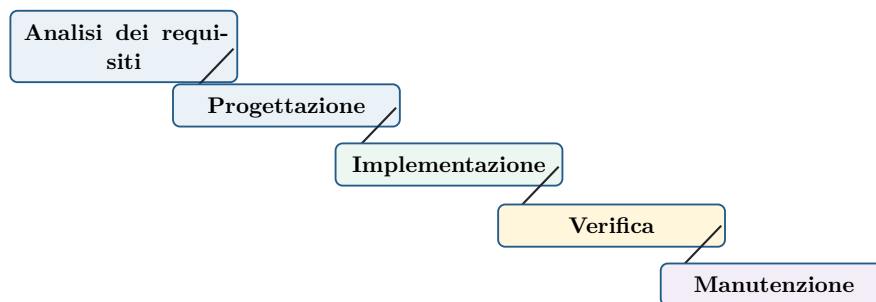


Figura 2: Nel Waterfall il lavoro procede per fasi sequenziali e consegne documentali.

Le fasi fondamentali sono:

1. **Analisi dei requisiti:** raccolta e documentazione delle necessità degli stakeholder.
2. **Progettazione:** definizione dell'architettura e delle soluzioni tecniche.
3. **Implementazione:** traduzione del progetto in codice.
4. **Verifica:** controllo del comportamento del sistema mediante test.
5. **Manutenzione:** correzioni e modifiche successive al rilascio.

1.4.2.1 Le indicazioni di Royce

Royce sottolinea cinque accorgimenti: progettare il programma prima della codifica, mantenere completa e aggiornata la documentazione, realizzare se possibile un prototipo preliminare ("fare il lavoro due volte"), pianificare e monitorare il testing fin dall'inizio e coinvolgere il cliente.

Punti di forza	Limiti
Disciplina, pianificazione ed enfasi su analisi e design.	Rigidità e scarso parallelismo tra le fasi.

Punti di forza	Limiti
Adatto a requisiti stabili e ben compresi.	Il cliente vede tardi il prodotto funzionante.
Artefatti e responsabilità chiaramente identificabili.	Le modifiche tardive sono costose e la documentazione può diventare eccessiva.

1.4.2.2 Studio di fattibilità

Prima dello sviluppo si valuta se il progetto sia conveniente e realizzabile. Lo studio considera costi, tempi, capacità tecniche, ritorno dell'investimento (**ROI**) e alternative come acquistare e personalizzare un prodotto esistente oppure svilupparlo da zero. Il risultato è una decisione **Go/No-go**.

1.4.2.3 Varianti

- **Cascata con prototipazione:** un prototipo “usa e getta” chiarisce requisiti e scelte prima del processo completo.
- **Cascata con feedback:** sono consentiti ritorni alle fasi precedenti quando emergono errori, attenuando la rigidità del modello puro.

1.4.3 V-Model

Il V-Model è una variante del Waterfall che rende esplicita la relazione tra ogni attività di analisi/progettazione e il corrispondente livello di test. Il codice occupa il vertice inferiore della V: a sinistra si scompone il problema, a destra si integra e verifica la soluzione.

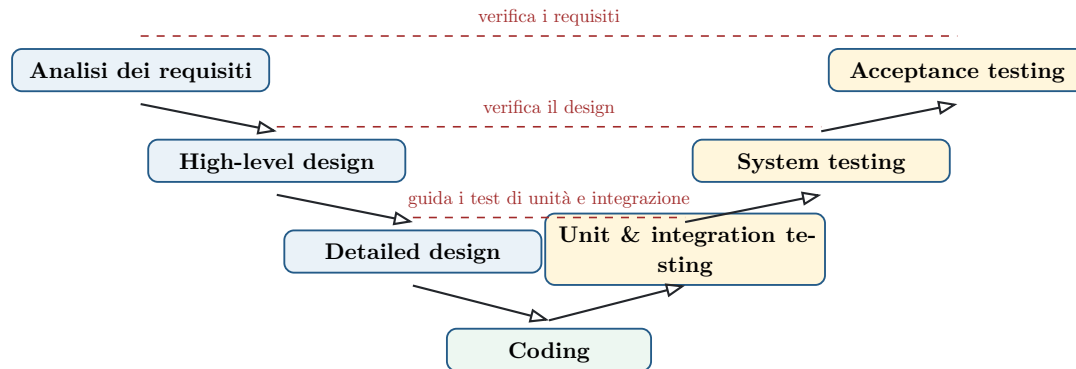


Figura 3: V-Model: ogni specifica preparata sul ramo sinistro è verificata dal test corrispondente sul ramo destro.

Analisi/design	Test corrispondente
Analisi dei requisiti	Acceptance testing con il cliente
High-level design	System testing sul sistema completo
Detailed design	Unit e integration testing

Le relazioni orizzontali hanno due direzioni concettuali. Le specifiche prodotte a sinistra **guidano** la preparazione dei test a destra; quando un test rivela un'incoerenza, il team può dover “mettere mano” alla specifica corrispondente. La preparazione dei test può quindi procedere in parallelo alle attività di analisi e design, anche se la loro esecuzione avviene dopo il coding.

Waterfall e V-Model soffrono quando i requisiti non possono essere congelati, quando il feedback arriva soltanto alla fine o quando errori iniziali si propagano fino alle fasi più costose.

1.4.4 Modelli evolutivi

Idea di base

Il prodotto non viene costruito in un'unica soluzione: si parte da un'implementazione parziale, la si espone agli utenti e la si raffina attraverso release successive.

I tre approcci principali sono prototipazione, sviluppo incrementale e sviluppo iterativo.

1.4.4.1 Prototipazione

Un prototipo è una versione anticipata del sistema, con meno funzionalità o qualità inferiore rispetto al prodotto finale. Può essere:

- **throwaway**: serve a ridurre l'incertezza e viene poi scartato;
- **evolutivo**: viene raffinato fino a diventare il prodotto finale.

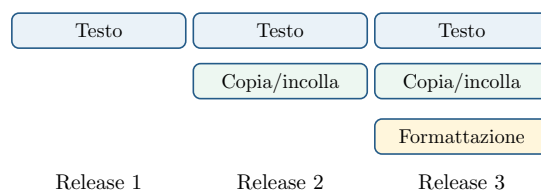
Il ciclo tipico comprende requisiti, quick design, costruzione del prototipo, valutazione del cliente e raffinamento. Conviene prototipizzare soprattutto le parti caratterizzate da requisiti vaghi, rischi tecnici o forti dubbi di usabilità.

Vantaggi	Rischi
Chiarisce requisiti incompleti e migliora la comunicazione.	Un prototipo da scartare ha un costo economico.
Fa emergere presto errori e rischi.	Cliente e team possono essere riluttanti a buttare codice funzionante.
Consente di validare interazione e scelte tecniche.	Scorciatoie e architettura debole possono finire nel prodotto.

1.4.4.2 Sviluppo iterativo-incrementale

Ogni release è pianificata, analizzata, progettata, implementata, testata e valutata dagli utenti. Mentre gli utenti lavorano con una release, il team può costruire la successiva.

Incrementale



Iterativo

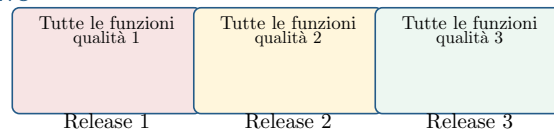


Figura 4: Incrementale: cresce l'insieme delle funzioni. Iterativo: cresce la qualità delle funzioni già presenti.

Nel modello **incrementale** ogni release aggiunge funzionalità senza eliminare quelle precedenti. Nel modello **iterativo** tutte le funzionalità principali sono presenti fin dall'inizio, ma vengono progressivamente migliorate. I due principi sono spesso combinati nello stesso progetto.

Le release possono dare precedenza alle funzionalità a rischio più alto oppure a quelle di maggior valore per l'utente. I vantaggi sono risposta rapida al mercato, feedback continuo, stime più gestibili e scoperta precoce degli errori.

1.4.5 Modello a spirale

Il modello a spirale, proposto da Barry Boehm nel 1988, è pensato per sistemi grandi e complessi. È **risk-driven**: il contenuto di ogni ciclo dipende dai rischi più importanti da affrontare. È anche un meta-modello, perché nel quadrante di engineering può impiegare Waterfall, prototipazione o altri processi.

Rischio

Un rischio è una circostanza sfavorevole che può compromettere progetto o prodotto. La sua esposizione si valuta combinando **probabilità** e **gravità delle conseguenze**: $E(R) = P(R) \times I(R)$.

Esempi sono tecnologie immature, componenti esterni inaffidabili e competenze insufficienti nel team.

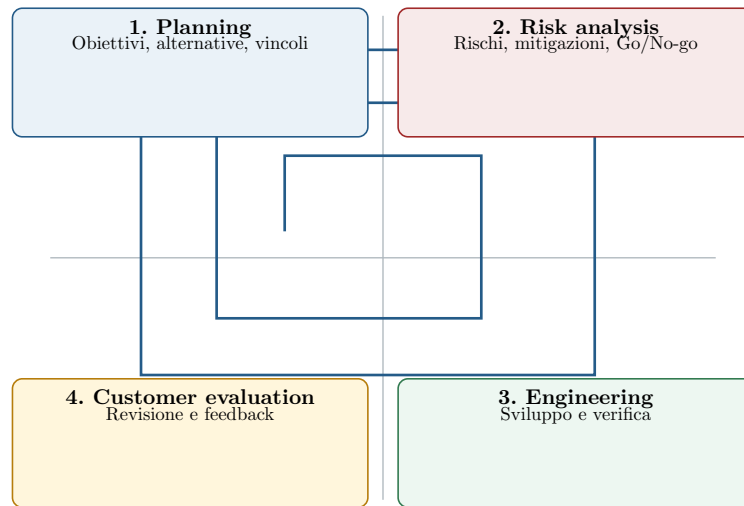


Figura 5: Ogni giro attraversa i quattro quadranti e amplia il prodotto e l'impegno economico.

Il modello affronta presto i problemi critici, ma richiede grande competenza nell'identificazione e nella stima dei rischi. Un rischio importante non riconosciuto può comunque compromettere il progetto.

1.4.6 Unified Process (UP/RUP)

Unified Process nasce dall'unificazione dei metodi di Grady Booch, James Rumbaugh e Ivar Jacobson, i "tre amigos" legati anche alla nascita di UML. La versione commerciale più nota è Rational Unified Process (RUP).

UP è orientato agli oggetti, guidato dai casi d'uso, basato su UML, iterativo e incrementale, attento ai rischi e fortemente plan-driven.

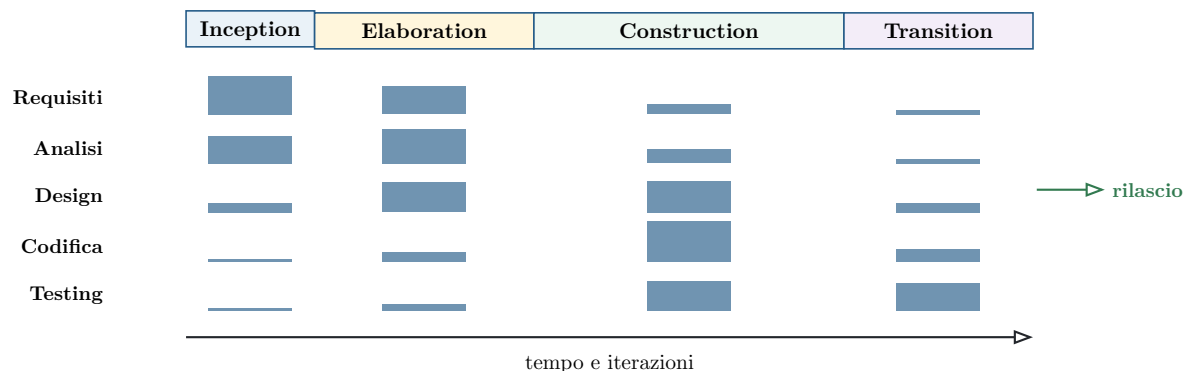


Figura 6: In UP tutte le discipline ricorrono, ma con intensità diversa nelle quattro fasi.

Le quattro fasi sono:

1. **Inception:** fattibilità, requisiti essenziali e rischi principali — “il progetto ha senso?”.
2. **Elaboration:** dominio, casi d'uso e architettura — “sappiamo che cosa costruire e come?”.
3. **Construction:** design di dettaglio, codice e test — “il software funziona?”.
4. **Transition:** deployment, formazione e accettazione — “gli utenti riescono a usarlo?”.

Ogni fase contiene una o più **iterazioni** e termina con una milestone. In ogni iterazione ricorrono requisiti, analisi, design, codifica e testing, ma la loro intensità cambia nel tempo. Il completamento della Transition conduce al rilascio del prodotto agli utenti.

1.4.7 Component-Based Software Engineering (CBSE)

CBSE costruisce sistemi componendo elementi software preesistenti e già testati. Il riuso può ridurre codice, costi e rischi, ma questi benefici dipendono dalla qualità dei componenti e dal costo d'integrazione.

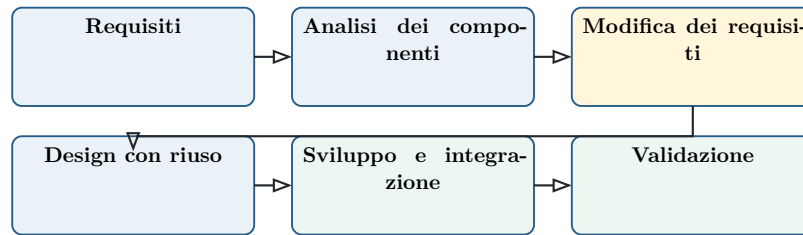


Figura 7: Nel CBSE anche i requisiti vengono negoziati in funzione dei componenti disponibili.

Durante l'analisi si cercano anzitutto componenti già disponibili; se non sono sufficienti, se ne possono acquistare di nuovi oppure sviluppare le parti mancanti. Il passaggio cruciale è la **modifica dei requisiti**: ciò che il cliente desidera deve essere conciliato con ciò che i componenti offrono. I limiti principali sono compromessi funzionali, difficoltà d'integrazione e perdita di controllo sull'evoluzione delle dipendenze esterne.

1.4.8 Modello trasformatore e metodi formali

Il modello trasformatore usa notazioni matematiche, come il linguaggio Z, per passare da requisiti informali a specifiche sempre più vicine all'esecuzione. Ogni trasformazione dovrebbe preservare la correttezza.

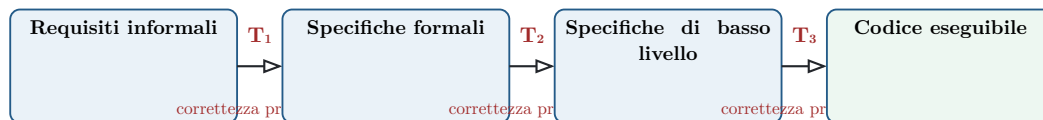


Figura 8: Catena di raffinamenti nel modello trasformatore.

Ogni trasformazione T_i prende una specifica e ne produce una più concreta, senza alterarne il significato previsto. In teoria, prove matematiche sui passaggi possono garantire proprietà molto forti. Il costo e la competenza richiesti ne limitano però l'uso a domini critici. Questo approccio anticipa il moderno Model-Driven Development, in cui modelli formali o semi-formali generano automaticamente parte del codice.

1.5 Approcci plan-driven e metodi agili

Aspetto	Plan-driven	Agile
Pianificazione	Prevalentemente anticipata	Progressiva e adattiva
Requisiti	Stabili e documentati	Evolgono col feedback
Misura del progresso	Piano e artefatti	Software funzionante
Cliente	Coinvolto in momenti definiti	Collaborazione frequente
Documentazione	Estesa e formale	Quanto basta al prodotto e al team
Esempi	Waterfall, V-Model, Spirale, UP	XP, Scrum

Gli approcci plan-driven privilegiano processi definiti, documentazione e controllo formale. Gli approcci agili nascono per rispondere rapidamente al cambiamento e dare priorità a software funzionante e collaborazione col cliente.

1.5.1 Pratiche di Extreme Programming

- iterazioni brevi di due-quattro settimane;
- design semplice secondo il principio KISS;
- refactoring continuo senza cambiare il comportamento osservabile;
- brevi meeting giornalieri;
- conoscenza condivisa soprattutto attraverso il team;
- cliente disponibile e vicino al gruppo di sviluppo.

1.5.2 Limiti dei metodi agili

Sul piano gestionale richiedono persone competenti e motivate; turnover elevato può disperdere conoscenza tacita. Sul piano legale è difficile fissare contratti quando ambito e risultato evolvono. Sul piano della manutenzione, documentazione insufficiente e architettura cresciuta senza controllo possono generare debito tecnico.

1.6 Come scegliere il modello

Preferire un approccio plan-driven quando...	Preferire un approccio agile quando...
Il sistema è grande, complesso o safety-critical.	Il team è piccolo, esperto e co-localizzato.
I requisiti sono stabili e verificabili in anticipo.	I requisiti sono volatili o ancora da scoprire.
Servono tracciabilità, certificazioni e documenti formali.	Gli utenti possono offrire feedback frequenti.
Il lavoro coinvolge molti team distribuiti.	È importante consegnare valore rapidamente.

Regola d'esame

La scelta è contestuale: dimensione, criticità, volatilità dei requisiti, disponibilità del cliente, distribuzione del team e capacità di gestione del rischio contano più dell'etichetta del modello. Nella pratica sono comuni processi ibridi.

1.7 Riepilogo

Primitivo	Code & Fix
Sequenziale	Waterfall, V-Model
Evolutivo	Prototipazione, incrementale, iterativo
Guidato dal rischio	Modello a spirale
Processo unificato	UP / RUP
Basato sul riuso	CBSE
Formale	Modello trasformatzionale
Adattivo	XP, Scrum, Kanban

Il filo conduttore è il compromesso tra pianificazione e adattamento. Più aumenta l'incertezza, più diventano importanti feedback e iterazioni; più aumentano criticità, scala e obblighi di conformità, più servono disciplina, tracciabilità e controllo esplicito.

2 Ingegneria dei requisiti

Un **requisito** descrive una proprietà che il sistema deve possedere: una funzione da offrire, un comportamento osservabile oppure un vincolo da rispettare. Può essere espresso a livelli di astrazione molto diversi, da un obiettivo generale in linguaggio naturale fino a una specifica dettagliata e matematica.

Principio fondamentale

Un requisito descrive principalmente **che cosa** il sistema deve fare, non **come** realizzarlo. Le decisioni implementative appartengono al design, salvo quando tecnologia, piattaforma o standard costituiscono essi stessi un vincolo richiesto.

Per esempio, per un ATM:

- “Il sistema deve controllare la validità della carta inserita” descrive una funzione;
- “Il sistema non deve erogare più di 250 euro per carta in un periodo di 24 ore” esprime una regola operativa misurabile.

2.0.1 Perché i requisiti sono decisivi

Una parte rilevante dei progetti fallisce per problemi che precedono la codifica. I dati riportati negli appunti mostrano che incompletezza dei requisiti e scarso coinvolgimento degli utenti occupano i primi posti.

Motivo di fallimento	Percentuale
Incompletezza dei requisiti	13,1%
Mancato coinvolgimento degli utenti	12,4%
Carenza di risorse	10,6%
Aspettative non realistiche	9,9%
Mancato supporto del management	9,3%
Cambiamento di requisiti e specifiche	8,7%
Carenza di pianificazione	8,1%
Cambiamento di obiettivi	7,5%
Carenza di gestione del settore IT	6,2%
Incompetenza tecnologica	4,3%
Altro	9,9%

2.0.2 Costo della scoperta tardiva

Un difetto introdotto nei requisiti diventa più costoso da correggere quanto più tardi viene scoperto: occorre modificare non soltanto la specifica, ma anche design, codice, test, documentazione ed eventualmente il sistema già distribuito. Validare presto i requisiti è quindi una strategia concreta di riduzione del costo e del rischio, non un adempimento burocratico.

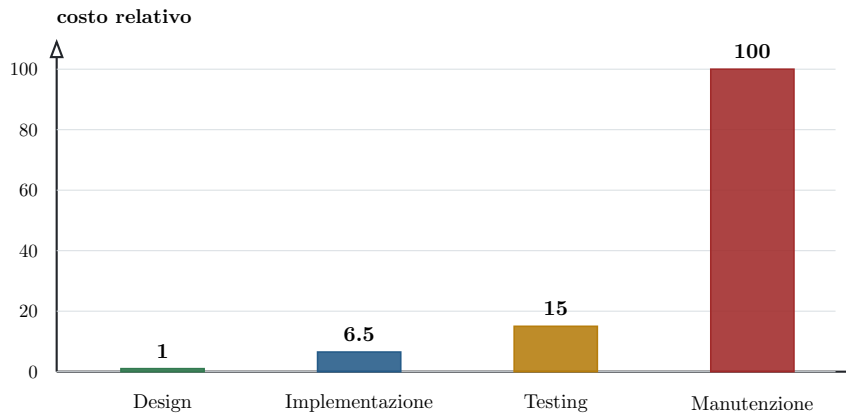


Figura 9: Costo relativo della rimozione di un difetto originato nei requisiti: 1 nel design, 6,5 nell'implementazione, 15 nel testing e 100 in manutenzione.

2.1 Livelli di dettaglio e rappresentazione

La documentazione procede normalmente da una descrizione comprensibile agli stakeholder verso una specifica abbastanza precisa da guidare progettazione, implementazione e test.

2.1.1 Requisiti utente

I **requisiti utente** descrivono in linguaggio naturale le funzionalità offerte e i principali vincoli operativi. Sono scritti per e con il cliente, devono essere comprensibili senza conoscenze tecniche e possono contribuire alla definizione contrattuale del prodotto.

Esempio per un editor grafico di modelli:

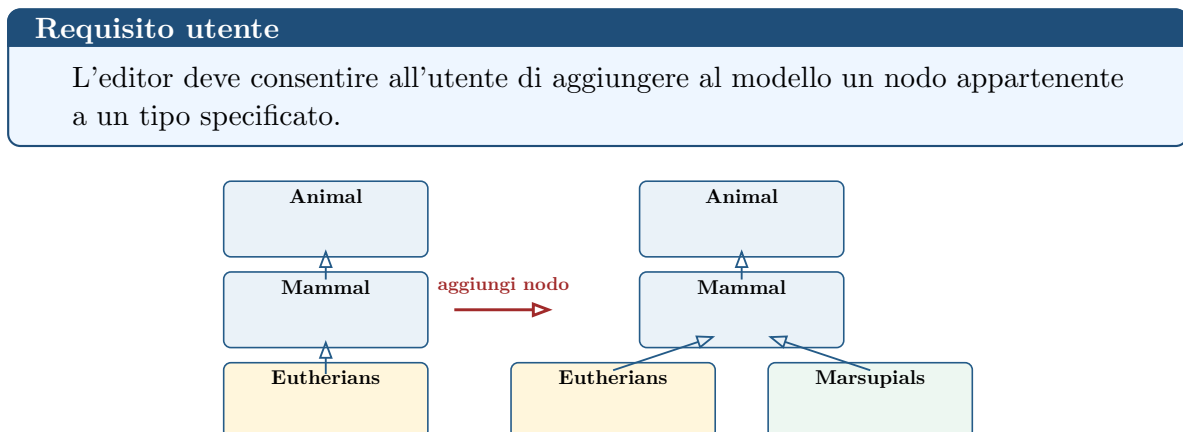


Figura 10: Il requisito utente esprime l'effetto osservabile: aggiungere al design un nuovo nodo e collegarlo correttamente alla gerarchia.

2.1.2 Requisiti di sistema

I **requisiti di sistema** traducono gli obiettivi utente in specifiche dettagliate e verificabili. Possono usare linguaggio naturale strutturato, modelli visuali o linguaggi formali. Nell'esempio precedente la funzione viene precisata così:

1. l'utente seleziona il tipo di nodo da aggiungere;
2. sposta il cursore nella posizione approssimativa del diagramma;
3. conferma l'inserimento del simbolo in quel punto.

Funzione	Aggiunta di un nodo
Descrizione	Aggiunge un nodo a un design esistente; l'utente ne sceglie tipo e posizione. Il nodo aggiunto diventa la selezione corrente.
Input	Tipo del nodo, posizione, identificatore del design.
Origine	Tipo e posizione dall'utente; identificatore dal database.
Output	Identificatore del design aggiornato.
Destinazione	Database dei design, aggiornato al completamento.
Richiede	Grafo del design associato all'identificatore ricevuto.
Precondizione	Il design è aperto e visualizzato.
Postcondizione	Il design è invariato salvo l'aggiunta del nodo specificato nella posizione scelta.
Effetti collaterali	Nessuno.

Dal requisito utente al requisito di sistema

Il primo comunica l'obiettivo agli stakeholder; il secondo elimina progressivamente ambiguità e fornisce dettagli sufficienti per costruire casi di test. I due livelli non sono concorrenti: servono destinatari diversi.

2.1.3 Modi di rappresentazione

I requisiti possono essere rappresentati con strumenti diversi, scelti in base a precisione richiesta, destinatari e costo di formalizzazione.

Forma	Caratteristiche
Nessuna specifica	Si passa direttamente all'implementazione: rapido ma rischioso e adatto soltanto a casi minimi.
Linguaggio naturale	Accessibile, ma può risultare ambiguo, incompleto o incoerente.
Testo strutturato	Template con campi obbligatori, precondizioni, input, output e postcondizioni.
Notazioni visuali	Diagrammi UML e altri modelli che rendono evidenti struttura e relazioni.
Use case	Descrivono le interazioni tra attori e sistema per raggiungere un obiettivo.
User story	Esprimono sinteticamente bisogno, soggetto e valore atteso.
Linguaggi formali	Notazioni matematiche, come Z, precise ma costose e specialistiche.

2.2 Requisiti funzionali e non funzionali

La distinzione fondamentale riguarda ciò che il sistema offre e le qualità o i vincoli entro cui deve operare.

2.2.1 Requisiti funzionali

I requisiti funzionali descrivono **funzioni e servizi**: ciò che il sistema deve fare, quali input tratta, quali risultati produce e come reagisce a determinate condizioni. Dovrebbero essere indipendenti dalla particolare soluzione implementativa.

Esempi per un sistema bancario:

- consentire la consultazione del saldo;
- permettere prelievo e produzione dell'estratto conto;
- richiedere, per il prelievo, l'autenticazione tramite un codice segreto memorizzato sulla carta.

2.2.2 Requisiti non funzionali

I requisiti non funzionali definiscono proprietà di qualità e vincoli sul prodotto o sul suo processo di sviluppo: prestazioni, affidabilità, usabilità, piattaforme, standard, formati, aspetti etici e obblighi legislativi.

Esempi:

- rispondere a un'interrogazione entro 3 secondi;
- produrre documenti in formato PDF;
- essere accessibile a persone con disabilità;
- funzionare su una piattaforma stabilita dall'organizzazione;
- essere facilmente espandibile per esigenze future.

Nell'esempio ATM, offrire prelievo, saldo ed estratto conto e autenticare il prelievo mediante codice segreto sono requisiti **funzionali**. Accessibilità, tempo di risposta, piattaforma ed espandibilità sono invece requisiti **non funzionali**.

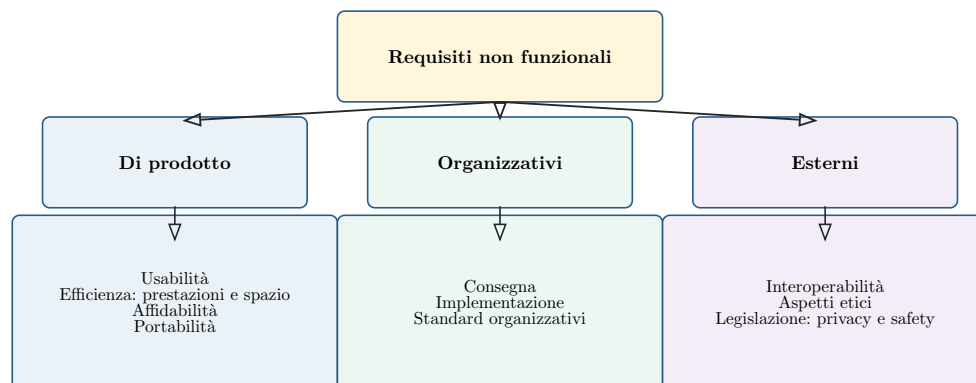


Figura 11: Tassonomia dei requisiti non funzionali: proprietà del prodotto, vincoli organizzativi e vincoli esterni.

Di prodotto	Qualità osservabili nel sistema: usabilità, efficienza, affidabilità e portabilità. L'efficienza comprende prestazioni e memoria occupata.
Organizzativi	Vincoli del processo e delle politiche aziendali: consegna, modalità d'implementazione, linguaggi, strumenti e standard.
Esterni	Vincoli del contesto: interoperabilità, principi etici, leggi, privacy e safety.

2.3 Qualità e verifica dei requisiti

Una checklist aiuta a controllare la specifica prima che gli errori si propaghino nelle fasi successive. Ogni requisito dovrebbe poter essere letto come un'affermazione controllabile, non come un'intenzione vaga.

2.3.1 Checklist essenziale

Necessario	Risponde a un bisogno reale di uno stakeholder o a un vincolo effettivo?
Chiaro	Ha una sola interpretazione ragionevole e usa termini definiti?
Completo	Specifica condizioni, soggetti, comportamento e risultato necessari?
Coerente	Non contraddice altri requisiti o regole del dominio?
Fattibile	Può essere realizzato con tecnologia, tempo e risorse disponibili?
Verificabile	È possibile dimostrare con test, misura, ispezione o analisi che è soddisfatto?
Tracciabile	Sono identificabili origine, motivazione e artefatti che lo implementano?
Prioritizzato	Importanza e urgenza sono note agli stakeholder?
Indipendente dalla soluzione	Evita dettagli tecnici non richiesti come veri vincoli?

Regola d'esame

Parole come “rapido”, “facile”, “adeguato” o “user-friendly” non rendono un requisito verificabile. Occorre sostituirle con criteri osservabili: per esempio “il 95% delle richieste deve ricevere risposta entro 3 secondi”.

2.3.2 Dalla specifica al test

Un requisito ben formulato anticipa il criterio con cui sarà accettato. Precondizioni, input, comportamento atteso, output e postcondizioni costituiscono quindi un ponte tra requirements engineering e testing.

Sintesi

I requisiti efficaci sono comprensibili agli stakeholder, sufficientemente precisi per gli sviluppatori e verificabili dai tester. La loro qualità determina la qualità delle decisioni prese in tutte le fasi successive.

3 Use case

Gli **use case** (casi d'uso), proposti da Ivar Jacobson nel 1992, esprimono i requisiti funzionali dal punto di vista di chi usa il sistema. Descrivono una storia di interazione orientata a uno scopo, senza entrare nella struttura interna o nella tecnologia impiegata.

Il “sistema” può essere un'organizzazione complessa, un prodotto software/hardware, una componente o un sottosistema. In ogni caso viene osservato come una **black box**: interessano i messaggi scambiati con l'esterno e il risultato visibile, non il modo in cui è costruito.

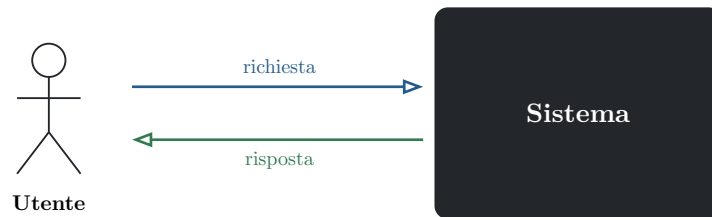


Figura 12: Un use case descrive l'interazione tra attori esterni e sistema attraverso uno scambio ordinato di messaggi.

Use case e diagramma UML

Lo use case è prima di tutto una **tecnica testuale**, indipendente dall'orientamento agli oggetti e da UML. Lo **Use Case Diagram** è invece una rappresentazione visuale UML che riassume attori, casi d'uso, relazioni e confine del sistema; non sostituisce la descrizione testuale.

3.0.1 Utilità nel ciclo di sviluppo

Uno use case collega il bisogno degli stakeholder agli artefatti successivi:

Requisiti	Esprime un obiettivo funzionale dal punto di vista dell'utilizzatore.
Design	Offre un punto di partenza per individuare responsabilità e componenti, senza imporre direttamente le classi.
Testing	Genera scenari e condizioni da trasformare in casi di prova.
Planning	Aiuta a raggruppare funzionalità e pianificare unità di rilascio.

3.1 Elementi di un modello di use case

Un modello completo combina quattro elementi: attori, casi d'uso, associazioni e confine del sistema.

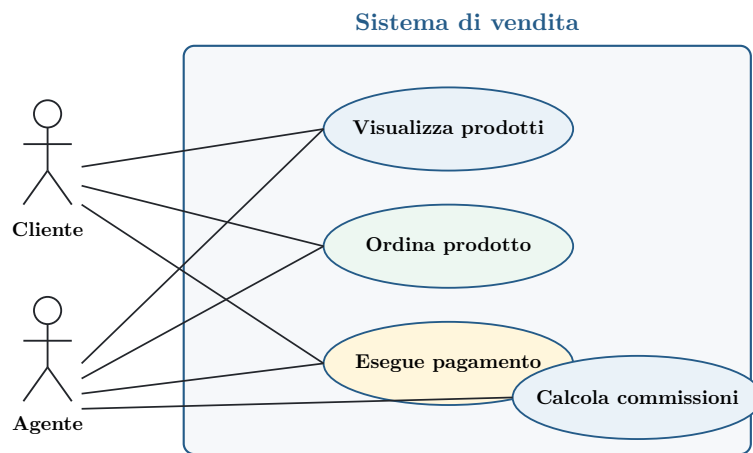


Figura 13: Esempio di Use Case Diagram: gli attori restano fuori dal confine, mentre i casi d'uso appartengono al sistema.

3.1.1 Confine del sistema

Il rettangolo di sistema stabilisce che cosa deve essere costruito e che cosa esiste già nell'ambiente. Il suo posizionamento modifica direttamente i requisiti funzionali: una responsabilità esterna non va specificata come comportamento interno e viceversa.

Errore frequente

Un confine ambiguo produce sovrapposizioni, funzionalità mancanti e aspettative incompatibili. Prima di descrivere gli use case bisogna chiarire sistemi esterni, persone, dispositivi e responsabilità organizzative.

3.1.2 Attori e ruoli

Un **attore** è il ruolo assunto da un'entità esterna durante l'interazione. Può essere una persona, un altro sistema o un dispositivo hardware. Non coincide con l'entità concreta: la stessa persona può agire, in momenti diversi, come cliente e amministratore; molte persone possono ricoprire lo stesso ruolo.

Attore primario	Persegue un obiettivo e ottiene valore dal sistema; normalmente avvia il caso d'uso. Esempio: il cliente che ordina un prodotto.
Attore secondario	Fornisce al sistema un servizio necessario. Esempio: il servizio esterno che autorizza un pagamento.

3.1.3 Associazioni

Una linea tra attore e caso d'uso indica **partecipazione**, non un flusso dati né necessariamente chi avvia l'interazione. Le associazioni rispondono alla domanda: "quali ruoli collaborano alla realizzazione di questo obiettivo?".

3.2 Scenari e descrizione testuale

Uno **scenario** è una sequenza ordinata di interazioni tra attori e sistema. Rappresenta una particolare esecuzione, cioè un singolo cammino, di uno use case.

Use case e scenario

Uno use case raccoglie tutti gli scenari che condividono lo stesso obiettivo finale. Per “prelevare contante” esistono il percorso di successo e percorsi alternativi: PIN errato, fondi insufficienti, ATM senza contante, richiesta di ricevuta o annullamento.

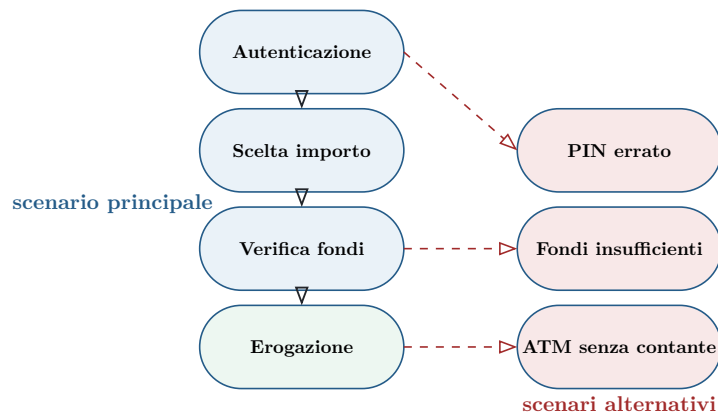


Figura 14: Uno use case comprende un percorso principale e più deviazioni o scenari alternativi.

Gli scenari concreti sono utili per scoprire i requisiti e per progettare i test; generalizzando più scenari si ottiene la descrizione complessiva dello use case.

3.2.1 Template di uno use case

Gli use case sono normalmente scritti con un template, usando il vocabolario del dominio e un linguaggio comprensibile al cliente.

Nome	Breve frase verbale attiva che esprime l'obiettivo, per esempio "Preleva contante".
Identificatore	Codice univoco, spesso numerico e progressivo.
Breve descrizione	Paragrafo che fissa lo scopo e il risultato atteso.
Attori primari	Ruoli che perseguono l'obiettivo.
Attori secondari	Ruoli o sistemi che forniscono servizi.
Precondizioni	Proprietà che devono essere vere prima dell'avvio.
Scenario principale	Sequenza nominale di passi che conduce al successo.
Postcondizioni	Proprietà vere dopo il completamento corretto.
Scenari alternativi	Elenco delle deviazioni documentate separatamente.

3.2.2 Esempio: prelevare contante

Caso d'uso: Preleva contante **Attore primario:** Cliente **Precondizioni:** collegamento bancario attivo; ATM disponibile e dotato di contante.

Scenario principale

1. Il cliente inserisce la carta.
2. Il sistema legge la carta e chiede il PIN.
3. Il cliente inserisce il PIN.
4. Il sistema convalida il PIN e mostra le operazioni disponibili.
5. Il cliente seleziona "Prelievo" e indica l'importo.
6. Il sistema verifica fondi e limiti giornalieri.
7. Il sistema autorizza l'operazione, aggiorna il conto ed eroga il contante.
8. Il sistema restituisce la carta e, se richiesta, stampa la ricevuta.

Postcondizione: il conto riflette il prelievo e la sessione è conclusa.

3.2.3 Regole per i passi

I passi devono essere concisi, numerati e ordinati temporalmente. Il formato consigliato è: "<numero> Il <soggetto> <azione>". Ogni passo attribuisce un comportamento osservabile al sistema oppure a un attore.

Livello corretto di dettaglio

Scrivere "Il cliente preme il pulsante OK" congela una scelta di interfaccia. È preferibile "Il cliente conferma l'ordine" oppure "Il sistema chiede al cliente di confermare l'ordine".

3.3 Deviazioni, ripetizioni e scenari alternativi

Una **deviazione** si verifica quando l'esecuzione si allontana dal percorso principale.

3.3.1 Deviazioni semplici e complesse

- Una deviazione breve può essere espressa inline con **se/altrimenti**.
- Una deviazione articolata, tipicamente un errore o un caso particolare che non rientra subito nel percorso principale, va documentata come scenario alternativo separato.

Estratto: Trova libro

1. Il cliente seleziona “Trova libro”.
2. Il sistema richiede i criteri di ricerca.
3. Il cliente inserisce i criteri e conferma.
4. Il sistema ricerca i libri corrispondenti.
5. **Se** trova uno o più libri, mostra l'elenco.
6. **Altrimenti**, comunica che non sono stati trovati risultati.

3.3.2 Ripetizioni

Le ripetizioni devono essere esplicite:

- **Per ogni** esprime un'iterazione su un insieme: per ogni libro trovato, il sistema mostra immagine, caratteristiche e prezzo.
- **Fintantoché** esprime un ciclo condizionato: fintantoché i criteri non sono validi, il sistema li richiede nuovamente e li convalida.

3.3.3 Template di uno scenario alternativo

Uno scenario alternativo possiede un identificatore derivato dallo use case principale (per esempio 9.1), breve descrizione, attori, precondizioni, sequenza e postcondizioni. Il primo passo specifica il punto d'innesto:

- “inizia dopo il passo X” quando può attivarsi soltanto in un punto preciso;
- “inizia in qualunque momento” per eventi sempre disponibili, come l'annullamento.

Nel template principale si elencano soltanto i nomi delle alternative, per esempio PINNonValido, FondiInsufficienti e Annulla. Identificatore e punto d'innesto rendono esplicita la relazione con il flusso principale.

3.4 Relazioni nel diagramma UML

Le relazioni strutturano il modello e riducono duplicazioni. Devono essere usate quando migliorano la comprensione, non come decorazione.

3.4.1 Inclusione (**include**)

L'inclusione estrae un comportamento obbligatorio e spesso comune a più use case. Il caso principale trasferisce il controllo al caso incluso e lo riprende al termine, in modo simile a una chiamata di procedura.

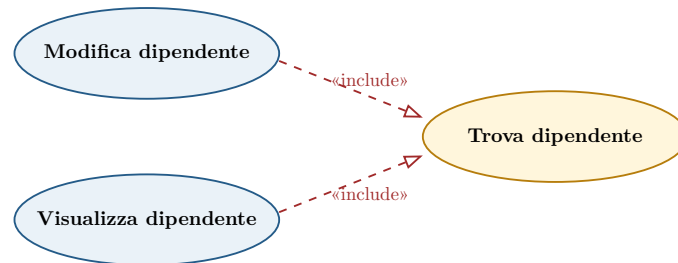


Figura 15: Trova dipendente è un comportamento comune incluso obbligatoriamente dai casi principali.

Semantica dell'include

Il caso principale è incompleto senza il caso incluso. Il caso incluso può essere un obiettivo autonomo oppure soltanto un frammento riutilizzabile.

3.4.2 Estensione (**extend**)

L'estensione aggiunge un comportamento opzionale a un caso base. Il caso base dichiara uno o più **extension point**, ma resta completo e comprensibile anche senza conoscere le estensioni.

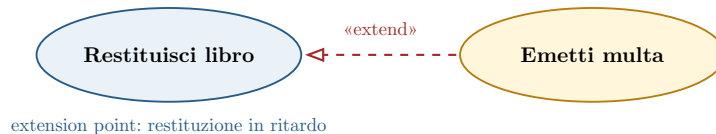


Figura 16: Emetti multa estende opzionalmente Restituisci libro quando la restituzione è in ritardo.

include	extend
Comportamento sempre richiesto.	Comportamento condizionale o opzionale.
Il principale dipende dall'incluso.	Il caso base è completo da solo.
La freccia punta al caso incluso.	La freccia punta al caso base.

3.4.3 Generalizzazione di attori

Un attore specializzato eredita tutte le associazioni dell'attore generale. Per esempio **Cliente** e **Agente** possono specializzare **Acquirente**, evitando di ripetere le relazioni comuni.

3.4.4 Generalizzazione di use case

Un caso figlio rappresenta una variante più specifica del genitore: eredita il comportamento, può aggiungere passi e può ridefinire quelli ereditati.

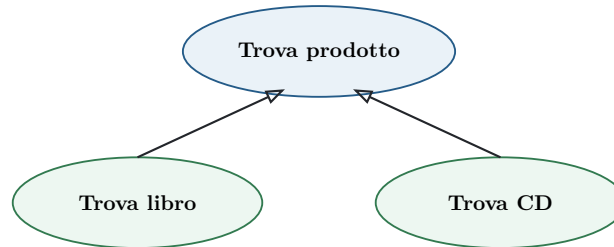


Figura 17: I casi specializzati ereditano l'obiettivo e il comportamento comune del caso generale.

Cautela

Le convenzioni per indicare passi ereditati, aggiunti e ridefiniti possono essere difficili per stakeholder non tecnici. Generalizzazione ed estensione vanno introdotte soltanto se rendono il modello più semplice.

3.5 Mockup, qualità della scrittura e limiti

3.5.1 Screen mockup

Uno use case può essere accompagnato da uno **screen mockup**: uno schizzo dell'interfaccia in uno specifico passo dello scenario. È utile nella negoziazione col cliente, guida lo sviluppatore e chiarisce le decisioni di UI, ma richiede tempo per essere prodotto e mantenuto.

I risultati sperimentali riportati negli appunti attribuiscono ai mockup un miglioramento del 69% nella comprensione degli use case e dell'89% nell'efficienza dei task di comprensione. Possono essere realizzati su carta o con strumenti di prototipazione come Pencil, usando stencil grafici per widget desktop, web e mobile.

3.5.2 Regole di buona scrittura

- mantenere scenario principale e use case brevi e semplici, idealmente entro una pagina A4;
- scegliere un livello intermedio: né troppo astratto né legato a dettagli inutili;
- descrivere interazioni osservabili, non funzionalità interne o algoritmi;
- usare il vocabolario del dominio e frasi comprensibili agli stakeholder;
- distinguere sempre il **cosa** dal **come**;
- evitare la decomposizione funzionale in casi astratti privi di valore autonomo;
- usare **include**, **extend** e generalizzazione soltanto quando semplificano davvero.

Principio guida

Gli use case sono documenti destinati a esseri umani. Precisione e struttura devono migliorare la comprensione, non trasformare la specifica in un esercizio di notazione UML.

3.5.3 Punti di forza e limiti

Punti di forza	Limiti
Esprimono requisiti funzionali in forma narrativa e verificabile.	Catturano male requisiti non funzionali trasversali.
Sono diffusi nei sistemi informativi e nello Unified Process.	Sono meno efficaci in sistemi dominati da algoritmi o con interazione esterna limitata.
Guidano design, pianificazione dei rilasci e testing.	Diagrammi senza specifiche testuali restano superficiali.
Template diversi mantengono una struttura sostanzialmente equivalente.	Un livello di dettaglio errato produce testi vaghi o vincolati prematuramente alla UI.

Regola d'esame

Uno use case non coincide con una funzione interna e non coincide con una singola schermata: descrive un obiettivo dell'attore realizzato attraverso più interazioni osservabili.

3.5.4 Riepilogo

Use case, scenari e diagrammi hanno ruoli complementari: il caso d'uso raccoglie i percorsi orientati a un obiettivo; lo scenario rappresenta un singolo percorso; il diagramma UML offre una mappa sintetica di attori, obiettivi e relazioni. La specifica testuale resta la fonte principale del comportamento richiesto.

4 Design architetturale

Il **design** è il processo che trasforma un problema in una soluzione. I requisiti definiscono il **cosa**; il design sceglie il **come** e struttura la soluzione; l'implementazione rende quella struttura eseguibile e utilizzabile.

L'analogia edilizia è utile: le richieste dei proprietari corrispondono ai requisiti, il progetto dell'architetto al design e i lavori di costruzione all'implementazione.

Due livelli di raffinamento

L'**architectural design** o high-level design identifica architettura, sottosistemi e comunicazioni. Il **component design** o low-level design dettaglia componenti, dati, algoritmi e tecnologie, passando eventualmente da una soluzione indipendente dalla piattaforma a una specifica per .NET, Jakarta EE o altri ambienti.

4.0.1 Creatività e riuso

Il design non è un'attività puramente creativa. Gran parte dei problemi viene risolta adattando conoscenza esistente:

Clonazione	Riuso quasi completo di design o codice, con modifiche limitate.
Design pattern	Soluzione generale a un problema ricorrente, applicabile in domini diversi.
Stile architetturale	Organizzazione generica di componenti e connettori, spesso ottimizzata per una famiglia di sistemi.

Buoni progetti precedenti, sistemi simili, modelli di riferimento, principi, convenzioni e pattern alimentano le decisioni del progettista. Studiare esempi riusciti migliora quindi la capacità di design.

4.1 Architettura software e attributi di qualità

Il **design architetturale** identifica le macro-componenti — moduli, sottosistemi, servizi o classi — e il framework di controllo e comunicazione che le collega. Il risultato è una descrizione dell'architettura software. Mary Shaw e David Garlan hanno contribuito a formalizzare questo campo negli anni Novanta.

4.1.1 Perché esplicitare l'architettura

- guida lo sviluppo e permette di affrontare componenti più semplici di un monolite;
- documenta responsabilità e dipendenze;
- supporta decisioni tecniche e manageriali;
- consente di analizzare proprietà come efficienza e manutenibilità;
- favorisce riuso su larga scala ed evoluzione controllata.

4.1.2 Componenti e connettori

Un diagramma architetturale a blocchi mostra un overview: i blocchi rappresentano componenti, mentre i connettori rappresentano invocazioni, pipe, eventi, scambio di messaggi o accesso a dati condivisi. UML offre il diagramma delle componenti; esistono anche Architecture Description Languages come Darwin e altri ADL.

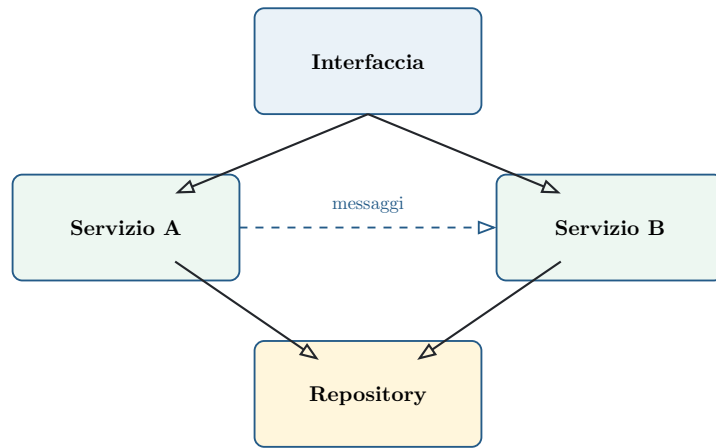


Figura 18: Componenti e connettori descrivono struttura e interazioni principali del sistema.

4.1.3 Impatto sugli attributi di qualità

Proprietà	Strategia architetturale tipica
Performance	Ridurre comunicazioni e usare componenti large-grain.
Security	Adottare livelli e collocare le risorse critiche negli strati interni.
Safety	Concentrare le funzionalità safety-critical in pochi sottosistemi controllabili.
Availability	Aggiungere ridondanza, replica e meccanismi di fault tolerance.
Maintainability	Preferire componenti fine-grain, coesi e sostituibili.

Trade-off inevitabili

Componenti grandi riducono le comunicazioni ma sono più difficili da sostituire; la ridondanza aumenta la disponibilità ma complica safety e consistenza; i livelli migliorano isolamento e security ma aggiungono attraversamenti e latenza. Non esiste un'architettura che massimizzi simultaneamente ogni qualità.

4.2 Stili architetturali strutturali

Uno **stile architetturale** definisce tipi di componenti, tipi di connettori e vincoli di composizione. È un modello generico da istanziare e personalizzare. I grandi sistemi reali sono spesso eterogenei e combinano più stili.

Gli stili **strutturali** evidenziano soprattutto chi comunica con chi; quelli **di controllo** mostrano anche chi determina l'ordine dell'esecuzione.

4.2.1 Layered architecture

Il sistema è organizzato in strati. Ogni livello offre servizi a quello superiore e usa normalmente soltanto il livello immediatamente inferiore, nascondendo i propri dettagli implementativi.

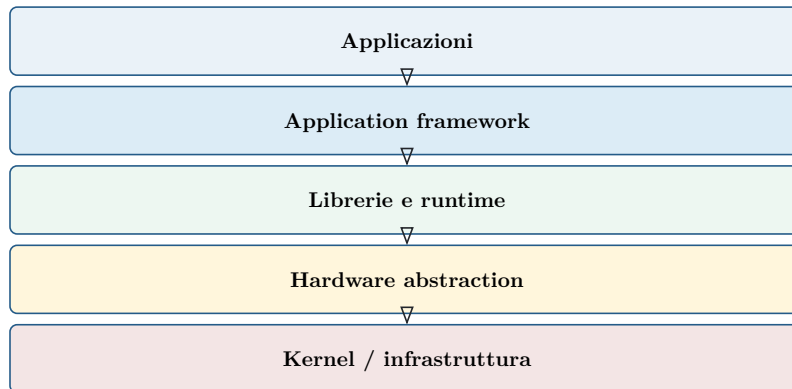


Figura 19: Architettura layered: ogni strato astrae i servizi dello strato sottostante. Android è un esempio noto di organizzazione multilivello.

Vantaggi	Svantaggi
Cambiamenti locali e sostituzione di implementazioni dietro un'interfaccia stabile.	La separazione può essere artificiale e generare molti attraversamenti.
Information hiding e riuso dei livelli.	Gli shortcut tra livelli migliorano talvolta le prestazioni ma rompono l'isolamento.

4.2.2 Repository

I sottosistemi condividono un deposito dati centrale e non comunicano direttamente. È adatto quando più strumenti producono e consumano grandi quantità di informazioni comuni, come nei CASE tool.

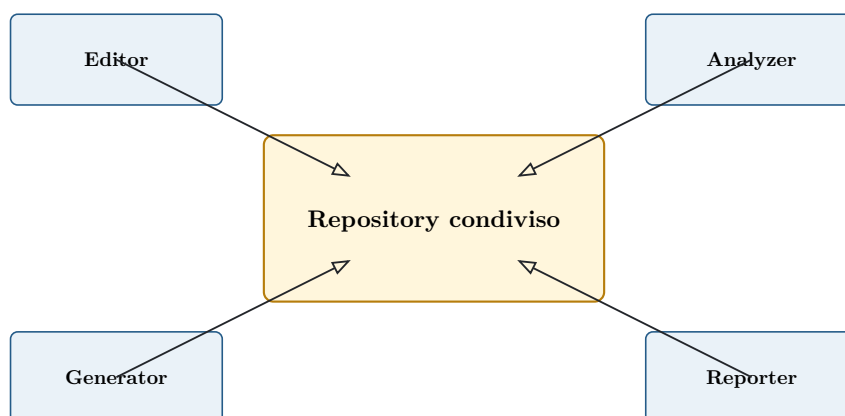


Figura 20: Nel repository model il deposito centrale media ogni condivisione dei dati.

Vantaggi	Svantaggi
Condivisione efficiente e gestione centralizzata di backup e sicurezza.	Schema comune rigido, evoluzione costosa e distribuzione difficile.

4.2.3 Client-server

I server offrono servizi specifici, i client li richiedono attraverso una rete e il protocollo segue tipicamente lo schema request/response.

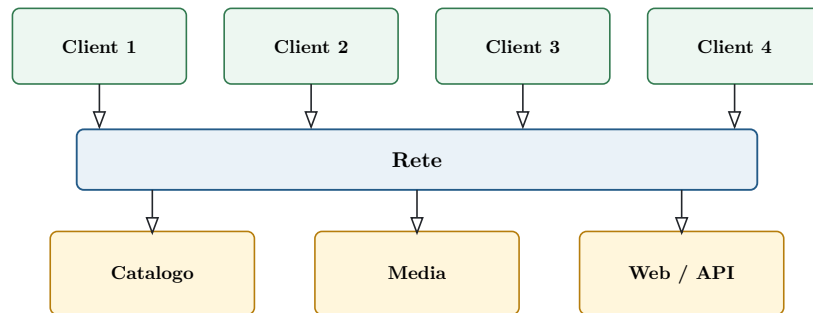


Figura 21: Client e server sono ruoli distinti collegati dalla rete.

Vantaggi	Svantaggi
Server riutilizzabili, scalabilità e possibilità di spostare computazione.	Modelli dati eterogenei, gestione duplicata e dipendenza da organizzazioni diverse.

4.2.3.1 Two-tier e three-tier

Two-tier	Three-tier
Presentation e business logic sono ripartite tra client e server. Un thin client mostra soltanto la UI; un fat client esegue anche elaborazione.	Client per la presentation, application server per la business logic e DB server per la data logic.
Più semplice, ma il client può diventare pesante e difficile da distribuire.	Separa chiaramente presentazione, logica e dati e consente di scalarli indipendentemente.

4.2.4 Peer-to-peer (P2P)

Ogni peer è contemporaneamente client e server: offre servizi e ne richiede ad altri. I ruoli non sono fissi e i nodi si distinguono soprattutto per i dati o le capacità possedute.

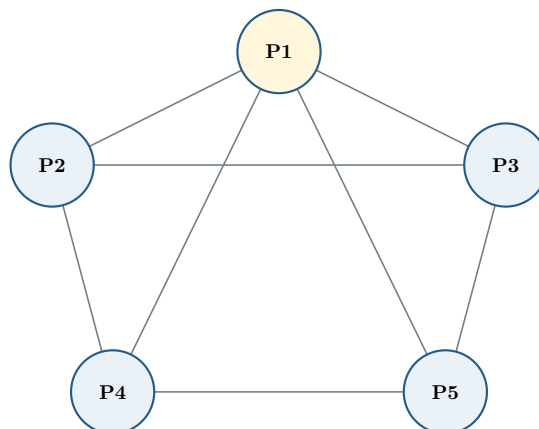


Figura 22: In una rete P2P i nodi collaborano senza una distinzione permanente tra client e server.

Replica e assenza di un singolo server favoriscono scalabilità e fault tolerance: aggiungere un peer può aggiungere dati e capacità. BitTorrent, eMule e reti blockchain sono esempi noti; coordinamento, consistenza e sicurezza restano però complessi.

4.2.5 Pipe and filter

I **filtri** trasformano flussi di input in output; le **pipe** trasportano i dati al filtro successivo.



Figura 23: Pipeline di trasformazioni indipendenti collegate da flussi dati.

L'esempio UNIX `cat input.txt | grep "Italy" | sort > output.txt` compone sorgente, filtri e destinazione. I filtri possono lavorare in parallelo quando producono e consumano il flusso progressivamente.

Vantaggi	Svantaggi
Riuso, composizione intuitiva, facile aggiunta di trasformazioni e concorrenza.	Poco adatto all'interazione; spesso batch; parsing e serializzazione possono costare.

4.3 Stili architetturali di controllo

Gli stili di controllo descrivono come viene determinato l'ordine dell'esecuzione. Nel controllo **centralizzato** un sottosistema avvia e coordina gli altri; nel controllo **event-based** i componenti reagiscono a eventi senza conoscere direttamente i destinatari.

4.3.1 Call-return

Il programma principale, o **driver**, chiama routine e sottoroutine secondo una gerarchia top-down. Il controllo passa alla routine chiamata e ritorna al chiamante al termine; un solo ramo è attivo alla volta, perciò il modello è adatto soprattutto a sistemi sequenziali.

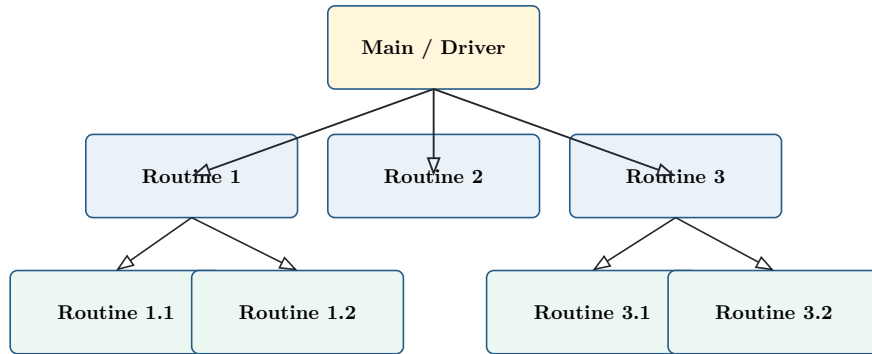


Figura 24: Call-return: il controllo discende dalla radice e ritorna lungo la gerarchia di chiamate.

4.3.2 Manager model

Un manager centrale avvia, termina e coordina processi che possono operare in parallelo. Interroga continuamente sensori, interfaccia e risultati di computazione, quindi attiva processi di calcolo e attuatori.

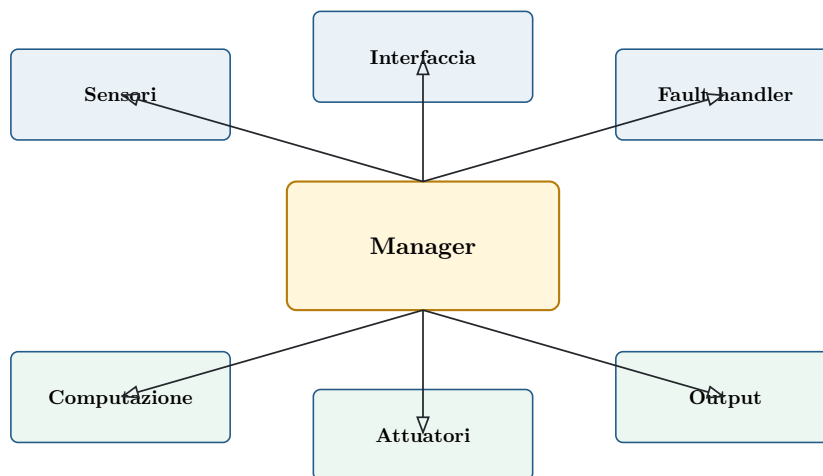


Figura 25: Il manager coordina processi concorrenti e dispositivi del mondo esterno.

È particolarmente adatto a sistemi real-time di controllo e guida. Un **attuatore** traduce un segnale di controllo in un'azione fisica.

4.3.3 Broadcast / publish-subscribe

Un componente pubblica un evento su un event bus; tutti i componenti sono in ascolto e quelli registrati reagiscono. Il produttore non invoca direttamente il consumatore: si parla di **implicit invocation**.

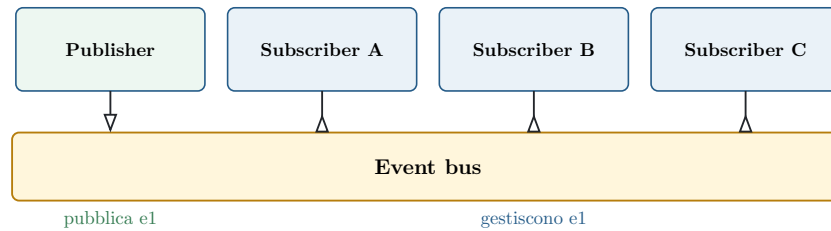


Figura 26: Nel publish-subscribe produttori e consumatori sono disaccoppiati dall'event bus.

Vantaggi	Svantaggi
Componenti sostituibili, estendibili e riusabili con dipendenze ridotte.	Gestione non deterministica, scambio dati complesso, performance e test più difficili.

4.3.3.1 Decisioni progettuali negli eventi

Il disaccoppiamento non elimina la necessità di un contratto: produttori e consumatori devono condividere nome, significato e struttura dell'evento. Occorre inoltre decidere come gestire duplicati, perdita e ordine dei messaggi.

Payload	I dati possono viaggiare con l'evento oppure essere recuperati da un repository mediante un identificatore.
Consegna	Il bus può garantire consegna al massimo una volta o almeno una volta; nel secondo caso i consumatori devono tollerare duplicati.
Ordinamento	Eventi concorrenti possono arrivare in ordine diverso da quello di pubblicazione.
Idempotenza	Elaborare più volte lo stesso evento non dovrebbe produrre effetti ulteriori indesiderati.
Osservabilità	Log, correlation ID e tracing aiutano a ricostruire flussi che attraversano componenti disaccoppiati.

Esempio

L'evento `OrdineConfermato` può attivare indipendentemente fatturazione, aggiornamento inventario e spedizione. Il servizio che conferma l'ordine non deve conoscere né invocare direttamente questi consumatori.

4.4 Architetture eterogenee

I sistemi reali raramente seguono uno stile puro. Gli stili possono essere combinati gerarchicamente — una componente interna adotta uno stile diverso da quello globale — oppure all'interno della stessa componente.

4.4.1 Esempio: compilatore

Un compilatore combina una pipeline di trasformazioni con un repository condiviso. Lexer, parser, analisi semantica, ottimizzazione e generazione del codice formano un pipe-and-filter; la symbol table è consultata da più fasi come repository.

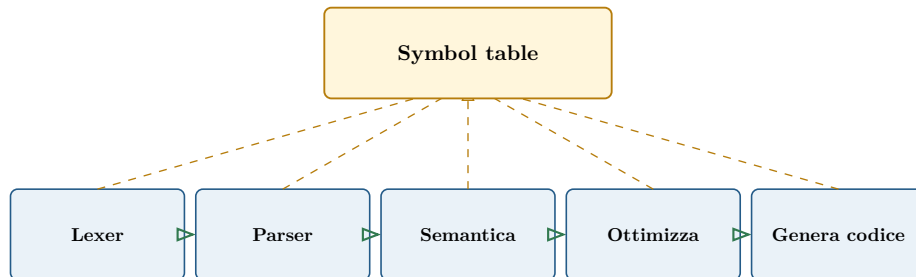


Figura 27: Architettura ibrida di un compilatore: pipeline di fasi e repository condiviso.

Criterio di scelta

Lo stile è uno strumento di ragionamento, non un vincolo ideologico. Componenti diverse possono richiedere organizzazioni diverse in funzione di dati, controllo e attributi di qualità.

4.5 Microservices

I microservizi evolvono le architetture service-oriented e contrastano il monolite, nel quale UI, business logic e accesso ai dati vengono distribuiti come un'unica applicazione.

4.5.1 Proprietà

- ogni servizio è indipendente e responsabile di una capacità di business coesa;
- servizi diversi possono usare tecnologie diverse;
- comunicano tramite API di rete, spesso HTTP/REST o messaggistica;
- possono essere distribuiti, scalati e aggiornati autonomamente.

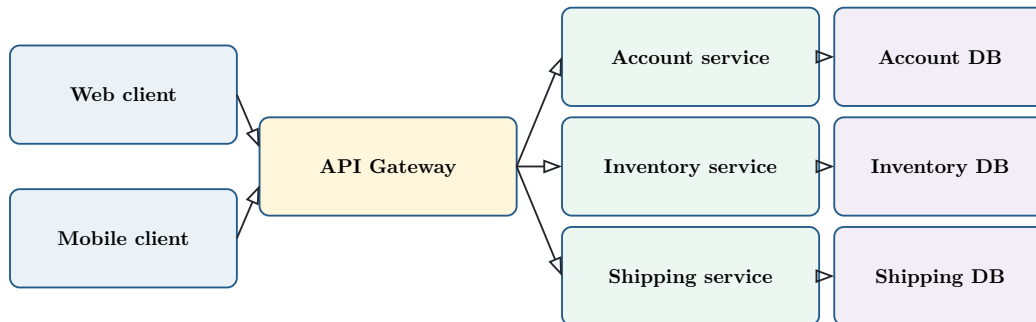


Figura 28: Il gateway espone un'interfaccia unificata e instrada le richieste verso servizi indipendenti.

4.5.2 API Gateway e REST

L'API Gateway offre ai client un endpoint unico, chiama i servizi necessari e aggrega le risposte. Riduce la conoscenza dell'architettura interna da parte dei client, ma può diventare un collo di bottiglia o un punto critico.

REST usa le operazioni HTTP per manipolare risorse:

CRUD	Metodo HTTP
Create	POST
Read	GET
Update	PUT / PATCH
Delete	DELETE

Vantaggi	Svantaggi
Scalabilità selettiva, deployment indipendente, fault isolation e libertà tecnologica.	Complessità distribuita, latenza di rete, test end-to-end e osservabilità difficili.
Team autonomi e servizi focalizzati su capacità specifiche.	Richiede infrastrutture per orchestrazione, monitoring, discovery e gestione dei dati distribuiti.

Docker e Kubernetes sono spesso usati per packaging e orchestrazione, ma non definiscono da soli un'architettura a microservizi.

4.6 Riepilogo e scelta dello stile

Layered	Separazione e information hiding tra livelli.
Repository	Condivisione attraverso un modello dati centrale.
Client-server	Ruoli stabili di fornitore e consumatore di servizi.
P2P	Peer simmetrici che offrono e richiedono servizi.
Pipe and filter	Trasformazioni indipendenti connesse da flussi.
Call-return	Controllo gerarchico e sequenziale.
Manager	Coordinamento centralizzato di processi concorrenti.
Broadcast	Reazione disaccoppiata a eventi pubblicati.
Microservices	Servizi distribuibili in modo indipendente attorno a capacità di business.

Sintesi

Requisiti → design architetturale → architettura software → possibile conformità a uno o più stili. La scelta deve partire dagli attributi di qualità e dai trade-off, non dalla popolarità di una tecnologia.

Il high-level design definisce la struttura globale; il low-level design dettaglia i componenti. Poiché progettare significa soprattutto selezionare, combinare e adattare soluzioni note, i sistemi reali risultano quasi sempre un mix ragionato di stili.

5 Design delle componenti (Low-Level Design)

Il Low-Level Design raffina l'architettura fino a descrivere componenti implementabili: classi, interfacce, attributi, operazioni, strutture dati e algoritmi. Se il High-Level Design stabilisce **quali sottosistemi** esistono e come comunicano, il Low-Level Design precisa **come è costruito ciascun sottosistema**.

5.0.1 Fasi del processo di design

Il processo non è rigidamente sequenziale: le attività si sovrappongono e possono consultare direttamente i requisiti. Ogni passaggio produce però un artefatto più preciso.

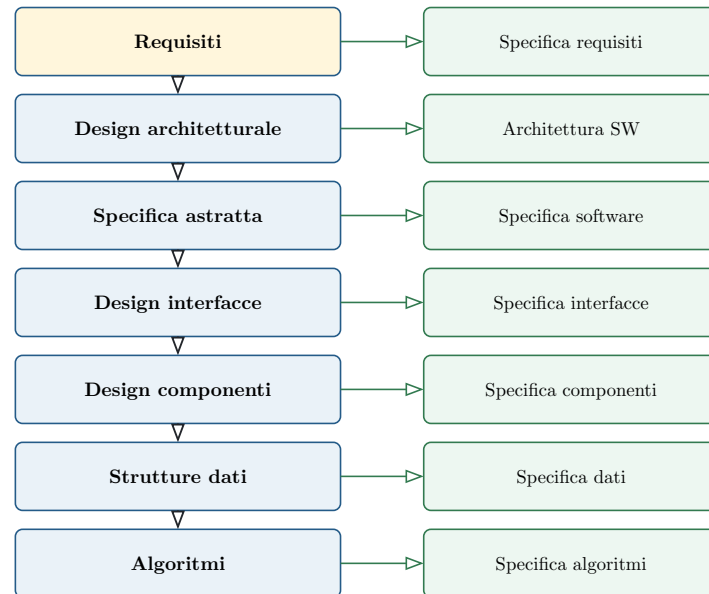


Figura 29: Raffinamento dalle esigenze agli algoritmi e relativi artefatti di design.

Architettura	Componenti, sottosistemi e connettori.
Specifica astratta	Responsabilità di alto livello dei componenti.
Interfacce	Servizi esposti, operazioni e dati scambiati.
Componenti	Classi, attributi, metodi e collaborazioni interne.
Strutture dati	Rappresentazioni adatte ai dati del problema.
Algoritmi	Procedimenti che implementano le operazioni.

Nel Weather Service, per esempio, l'architettura individua `WeatherService`, `LocalInformation`, `SocialNetwork` e `Subscriber`; il design delle interfacce definisce operazioni come `GetHistoricalTemperature`; il component/data design introduce attributi tipizzati; l'algorithm design seleziona ordinamenti e collezioni adeguate.

5.1 Design by Contract

Il **Design by Contract** (Bertrand Meyer, Eiffel, 1988) specifica l'interfaccia di una componente prima della codifica mediante un accordo preciso tra chi offre un servizio e chi lo usa. L'obiettivo è rendere espliciti obblighi, garanzie e responsabilità.

Metafora del contratto

Il **cliente** o caller deve rispettare le condizioni richieste dal servizio. Il **fornitore** o supplier garantisce il risultato soltanto quando tali condizioni sono soddisfatte.

5.1.1 Precondizione, postcondizione e invariante

Precondizione	Proprietà booleana che deve valere prima dell'operazione; è responsabilità del chiamante.
Postcondizione	Proprietà garantita dopo l'operazione se la precondizione era valida.
Invariante	Proprietà che ogni oggetto deve rispettare negli stati osservabili, prima e dopo ogni operazione pubblica.

Per una radice quadrata intera:

`sqrt(x)` **pre:** $x \geq 0$ **post:** $x = \text{result} \times \text{result}$

Durante l'esecuzione interna di un metodo l'invariante può essere temporaneamente violato, ma deve essere ripristinato prima che il controllo torni al cliente.

5.1.2 Esempio: conto corrente

Per `withdraw(value)`:

Precondition $\text{value} \geq 0 \text{ value} \leq \text{balance}$

Postcondition $\text{balance} = \text{balance@pre} - \text{value}$

Class invariant $\text{balance} \geq 0 \text{ balance} = \text{sum}(\text{deposits}) - \text{sum}(\text{withdrawals})$

`balance@pre` indica il saldo prima della chiamata. Il contratto garantisce che il prelievo modifichi esattamente il saldo e che l'oggetto resti coerente.

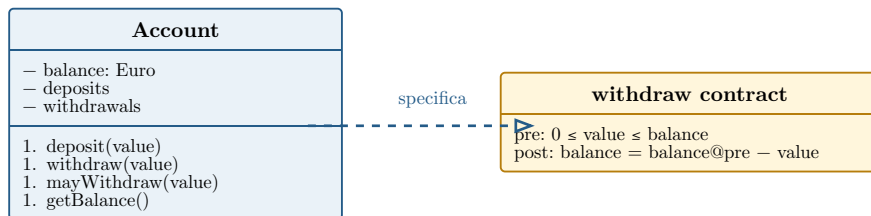


Figura 30: Classe Account e contratto dell'operazione `withdraw`.

5.1.3 Responsabilità del cliente e del fornitore

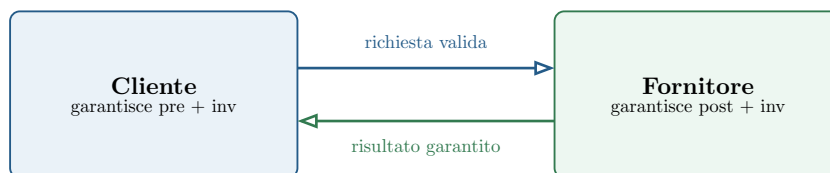


Figura 31: Il contratto distribuisce esplicitamente obblighi e garanzie.

Violazione	Responsabile
Precondizione non valida	Cliente/chiamante
Postcondizione o invariante non validi, con pre valida	Fornitore/implementazione

Senza questa attribuzione si rischiano troppo pochi controlli — ciascuno li delega all'altro — oppure controlli duplicati e incoerenti tipici di un defensive programming indiscriminato.

5.1.4 Vantaggi

- guida codifica e design delle operazioni;
- documenta il comportamento meglio di commenti informali;
- genera test black-box dalle pre/postcondizioni;
- localizza più rapidamente la responsabilità di un failure;
- favorisce operazioni semplici con responsabilità circoscritte.

Eiffel integra nativamente i contratti; altri ecosistemi offrono `assert` o librerie come JML, `icontract`, `deal` e `Code Contracts`.

5.2 Progettazione degli algoritmi

Il design degli algoritmi è la parte più vicina alla codifica. Si parte dalla descrizione della classe target, si riusa quando possibile un algoritmo noto e si progetta una soluzione nuova soltanto quando necessario.

Il percorso tipico è: selezione della notazione, raffinamento progressivo e — nei sistemi critici — prova formale della correttezza, per esempio mediante triple di Hoare.

5.2.1 Notazioni e PDL

Le notazioni visuali includono activity diagram, flowchart, box diagram, structured chart e decision table. Le notazioni testuali includono PDL e pseudocodice.

Il **Program Design Language** combina il vocabolario del linguaggio naturale con strutture di controllo simili a un linguaggio di programmazione. La narrativa consente di mantenere inizialmente un alto livello di astrazione.

```
SORT(TABLE, SIZE)
  IF SIZE > 1
    DO UNTIL NO ITEMS WERE INTERCHANGED
      FOR EACH ADJACENT PAIR IN TABLE
        IF FIRST ITEM > SECOND ITEM
          INTERCHANGE THE TWO ITEMS
```

5.2.2 Stepwise refinement

Lo stepwise refinement sostituisce progressivamente descrizioni narrative con istruzioni precise fino a ottenere una soluzione direttamente traducibile in codice.



Figura 32: Ogni raffinamento riduce l’astrazione e aggiunge decisioni verificabili.

Esempio: per calcolare la differenza tra massimo e minimo di tre interi si passa da “acquisisci A, B, C e calcola massimo e minimo” a chiamate `CALCOLA_MASSIMO/CALCOLA_MINIMO`, quindi si raffina ciascuna operazione con confronti e assegnamenti espliciti.

Quando fermarsi

Il raffinamento termina quando ogni passo è sufficientemente preciso da essere tradotto senza introdurre nuove decisioni sostanziali durante la codifica.

5.3 Principi di progettazione

I principi di design orientano verso software manutenibile, comprensibile, testabile, riusabile, riparabile e portabile. Sono indipendenti dal paradigma OO.

Un **modulo** è qualsiasi entità software nominata che contiene e offre servizi: funzione, script, classe, package, programma o sottosistema. Un modulo può includerne o usarne altri, creando dipendenze.

5.3.1 Astrazione

L'astrazione consente di concentrarsi sulle proprietà rilevanti a un livello, ignorando temporaneamente i dettagli. Riduce la complessità e rende possibile descrivere il comportamento senza fissarne subito l'implementazione.

Forma	Che cosa astrae	Esempio
Funzionale	Algoritmo che realizza un servizio.	<code>sort(list)</code>
Dati	Struttura concreta dietro operazioni pubbliche.	Stack: <code>push/pop</code>
Controllo	Meccanismo interno di coordinamento.	Semaforo

Nel Selection Sort si può passare da “ordina L” alla ricerca iterativa del minimo e infine ai cicli e scambi completi. Ogni livello risponde a domande diverse senza invalidare quelli più astratti.

5.3.2 Decomposizione

Risolvere sottoproblemi separati richiede in genere meno effort che affrontare l'intera complessità in una volta. Tuttavia, aumentando i moduli cresce anche il costo d'integrazione.

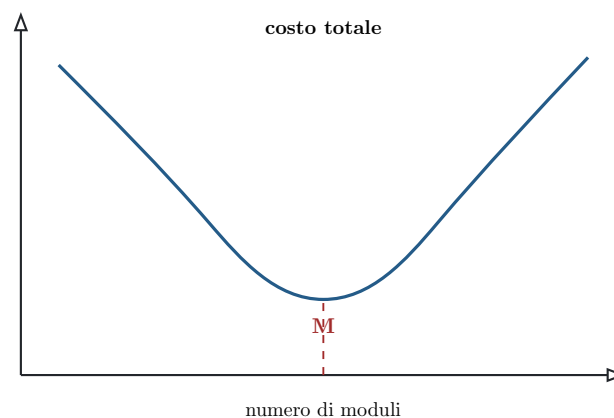


Figura 33: Esiste un numero di moduli M che bilancia complessità interna e costo d'integrazione.

Troppo pochi moduli concentrano la complessità; troppi moltiplicano interfacce, coordinamento e test d'integrazione. La decomposizione va quindi ottimizzata, non massimizzata.

5.3.3 Modularità e Separation of Concerns

La modularità aggiunge un criterio qualitativo alla decomposizione: ogni modulo dovrebbe occuparsi di un concern coerente e avere una sola ragione significativa per cambiare.



Figura 34: Separare calcolo e presentazione permette di sostituire la UI senza modificare la logica.

La stessa idea porta ai livelli presentation, business e data logic e a pattern come MVC.

Criteri fondamentali

Una buona modularizzazione **massimizza la coesione** interna e **minimizza l'accoppiamento** tra moduli. Moduli coesi e poco accoppiati sono più facili da comprendere, testare, modificare e riusare.

5.3.4 Information hiding

Ogni modulo espone l'interfaccia — ciò che offre — e nasconde le decisioni interne — come lo realizza. Finché il contratto pubblico rimane stabile, l'implementazione può cambiare senza effetti a cascata.



Figura 35: Il cliente dipende dall'interfaccia stabile, non dalle decisioni nascoste.

In Java e C++ l'information hiding è supportato da **private**, **protected** e **public** e dall'incapsulamento. Il beneficio generale è la riduzione dell'accoppiamento, non il semplice uso di modificatori sintattici.

5.4 Riepilogo

Design by Contract	Precondizioni, postcondizioni e invarianti assegnano responsabilità a cliente e fornitore.
Algorithm design	PDL e stepwise refinement portano dalla narrativa al codice.
Astrazione	Mostra ciò che conta al livello corrente.
Decomposizione	Divide il problema bilanciando complessità e integrazione.
Modularità	Massimizza coesione e minimizza accoppiamento.
Information hiding	Protegge le decisioni interne dietro interfacce stabili.

Dal problema al codice

Requisiti → High-Level Design (architettura e connettori) → Low-Level Design (classi, contratti, dati e algoritmi) → implementazione. Ogni raffinamento aggiunge precisione senza perdere la tracciabilità verso il problema originario.

6 Introduzione a UML

UML (**Unified Modeling Language**) è una famiglia unificata di notazioni grafiche per descrivere, progettare e documentare sistemi. È particolarmente diffusa per sistemi object-oriented, ma può modellare anche hardware, organizzazioni e domini di business. Lo standard è mantenuto dall'OMG (**Object Management Group**).

Che cosa UML non è

UML non è una metodologia di sviluppo, non è normalmente un linguaggio di programmazione e non è vincolato allo Unified Process. Può essere usato in processi plan-driven, iterativi o agili.

6.1 Modi d'uso e prospettive

6.1.1 Sketch, blueprint e linguaggio eseguibile

Sketch	Diagramma parziale per comunicare, discutere e chiarire le decisioni più importanti. Si realizza anche su lavagna.
Blueprint	Modello dettagliato che dovrebbe guidare quasi meccanicamente l'implementazione; richiede strumenti di modellazione e manutenzione rigorosa.
Linguaggio eseguibile	Modello sufficientemente formale da essere trasformato o eseguito da strumenti specifici.

Nella pratica lo sketch è l'uso più comune: seleziona l'informazione utile invece di rappresentare ogni dettaglio. Un blueprint completo può essere costoso, difficile da validare e rapidamente obsoleto.

6.1.2 Prospettiva concettuale e software

Elemento	Concettuale	Software
Classe	Concetto significativo del dominio.	Classe o modulo da implementare.
Oggetto	Entità concreta del mondo reale.	Istanza a runtime.
Operazione	Responsabilità o azione del dominio.	Servizio implementato da un metodo.
Scopo	Comprendere business e terminologia.	Progettare e documentare il software.

Dichiarare sempre la prospettiva

Lo stesso simbolo può avere significati diversi. Prima di leggere un diagramma bisogna sapere se rappresenta concetti del dominio oppure elementi implementativi.

6.1.3 Modello di dominio

Un modello di dominio è un glossario visuale delle astrazioni rilevanti. Non rappresenta tutto il mondo reale: seleziona concetti, attributi e relazioni utili allo scopo, crea un vocabolario comune tra stakeholder e ispira successivamente nomi e strutture del design.

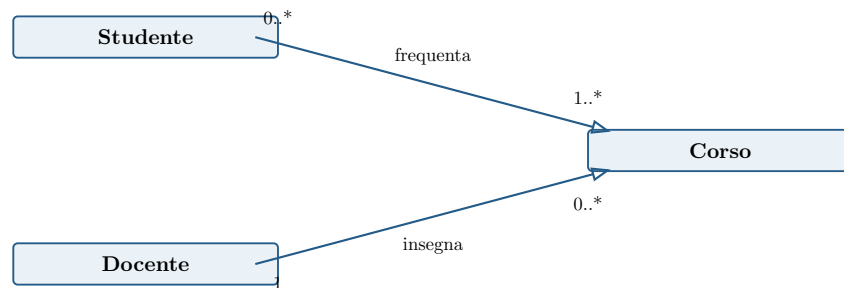


Figura 36: Modello concettuale del dominio universitario.

6.2 Notazione, meta-modello e viste

UML definisce sia una **notazione** sia un **meta-modello**. La notazione è la sintassi grafica — per esempio rettangoli, linee e molteplicità — mentre il meta-modello definisce i concetti del linguaggio, come Classe, Attributo, Operazione e Associazione.

Per uno sketch è spesso sufficiente conoscere bene la notazione; strumenti di modellazione, trasformazioni e profili richiedono invece maggiore attenzione alla semantica del meta-modello.

6.2.1 Viste e modello 4+1

Un sistema complesso viene descritto mediante più **viste**, ciascuna orientata alle esigenze di uno stakeholder. Un diagramma presenta una vista o una sua parte; un modello è l'insieme coordinato dei diagrammi.

Vista	Destinatario	Domanda principale
Logica	Analista/progettista	Quali funzionalità e astrazioni offre il sistema?
Sviluppo	Developer	Come è organizzato il codice?
Processi	Integrator	Come collaborano processi e thread?
Fisica	System engineer	Dove sono distribuiti software e hardware?
Scenari (+1)	Stakeholder	Come le viste realizzano casi d'uso significativi?

Viste sovrapposte possono essere incoerenti; viste incomplete possono lasciare decisioni scoperte. Completezza e consistenza vanno bilanciate con la leggibilità.

6.3 I diagrammi di UML 2.5

UML 2.5 comprende quattordici tipi di diagramma, divisi tra struttura e comportamento. I diagrammi di interazione sono un sottoinsieme di quelli comportamentali.

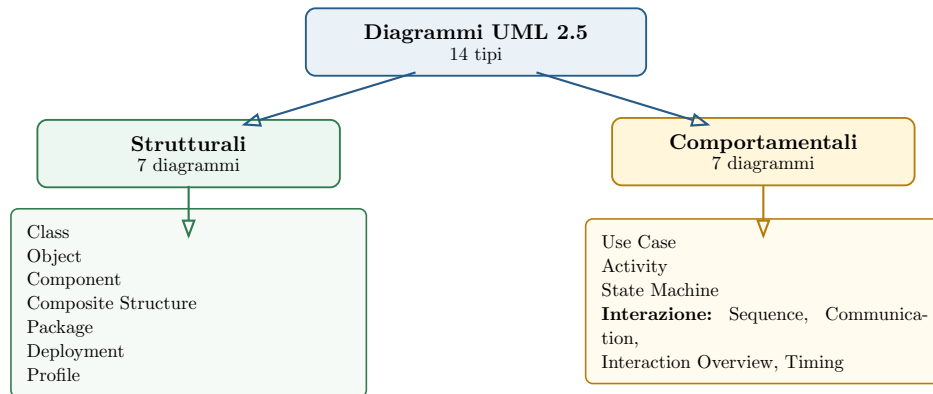


Figura 37: Famiglie dei quattordici diagrammi UML 2.5.

6.3.1 Diagrammi principali nel corso

- **Use Case:** obiettivi funzionali osservati dagli attori.
- **Class e Object:** struttura statica e istanze concrete.
- **Sequence e Communication:** flusso dei messaggi tra oggetti.
- **Activity:** flussi di lavoro e algoritmi.
- **State Machine:** comportamento reattivo dipendente dallo stato.
- **Component e Deployment:** organizzazione software e distribuzione sull'hardware.

Principio 80/20

Non è necessario usare ogni diagramma. Un piccolo sottoinsieme ben compreso copre la maggior parte delle esigenze di comunicazione e design.

6.4 UML nel processo di sviluppo

UML può accompagnare requisiti, architettura e design senza imporre un processo specifico.

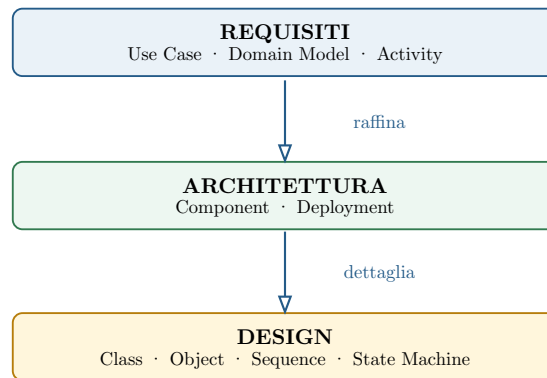


Figura 38: Collocazione tipica dei principali diagrammi nel processo.

I domain model chiariscono il vocabolario; i diagrammi dei componenti descrivono l'architettura; deployment collega nodi hardware e componenti software; class diagram e object diagram guidano il Low-Level Design; sequence e communication mostrano le collaborazioni; activity e state machine precisano algoritmi e comportamento dinamico.

6.5 Profili, codice e uso pragmatico

6.5.1 Profili UML

Un profilo adatta UML a un dominio o a una tecnologia mediante:

- **stereotipi**, scritti come «stereotype» o rappresentati da icone;
- vincoli aggiuntivi sugli elementi ammessi;
- semantica specifica del dominio.

Profili possono descrivere applicazioni web, sistemi embedded, domini ferroviari o spaziali. Sono estensioni controllate, non nuovi linguaggi scollegati da UML.

6.5.2 UML e codice

Non esiste una corrispondenza universale e completa tra UML e ogni linguaggio di programmazione. Un diagramma è normalmente più astratto del codice: offre una buona approssimazione della struttura o del comportamento, ma non determina ogni dettaglio esecutivo.

6.5.3 Consigli pratici

1. Dichiarare uso e prospettiva: per esempio “sketch software”.
2. Ricordare che ogni informazione può essere soppressa: l’assenza non dimostra che qualcosa non esista.
3. Preferire un diagramma chiaro e leggermente convenzionale a uno formalmente perfetto ma illeggibile.
4. Usare soltanto il sottoinsieme di UML realmente compreso e utile.
5. Segnalare le convenzioni non normative.
6. Non esitare a usare diagrammi non UML quando comunicano meglio lo scopo.

Regola fondamentale

La completezza è nemica della chiarezza. Lo scopo primario del diagramma è far comprendere una decisione o una struttura; la conformità allo standard serve la comunicazione, non la sostituisce.

6.6 Riepilogo

UML è una lingua franca visuale, non una metodologia. Può essere usato come sketch, blueprint o modello eseguibile e può assumere prospettiva concettuale o software. Un modello combina viste e diagrammi diversi; la loro scelta dipende dagli stakeholder, dalla fase del processo e dalla domanda a cui si vuole rispondere.

7 Class diagram e Object diagram UML

Il **Class Diagram** descrive la struttura statica di ciò che si sta modellando: classi, feature e relazioni. Le feature comprendono attributi e operazioni; le relazioni principali sono associazione, aggregazione, composizione, generalizzazione e dipendenza.

La prospettiva cambia il significato

In prospettiva concettuale una classe rappresenta un concetto del dominio e un'operazione una responsabilità. In prospettiva software la classe è un modulo implementabile e l'operazione è realizzata da uno o più metodi.

7.1 Classi, attributi e operazioni

Una classe incapsula caratteristiche comuni a un insieme di oggetti. Gli attributi determinano lo stato; le operazioni descrivono il comportamento disponibile. Oggetti collegati collaborano scambiando messaggi, cioè invocando operazioni.

7.1.1 Notazione della classe

La rappresentazione completa usa tre compartimenti: nome, attributi e operazioni. I compartimenti non necessari possono essere soppressi per mantenere il diagramma leggibile.

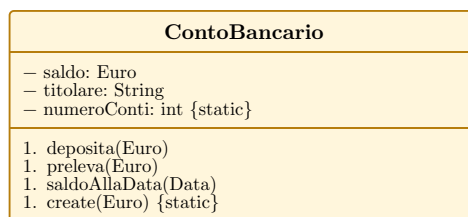


Figura 39: Classe UML con i compartimenti di nome, attributi e operazioni.

7.1.2 Attributi

La sintassi generale è:

visibilità nome: tipo [molteplicità] = default {proprietà}

Simbolo	Visibilità	Accesso
+	public	Tutte le classi.
-	private	Soltanto la classe proprietaria.
#	protected	Classe e sottoclassi.
~	package	Classi dello stesso package.

Solo il nome è obbligatorio. Il tipo può essere primitivo, datatype o classe del modello. Default indica il valore iniziale; proprietà come `{readOnly}`, `{ordered}`, `{static}` o `{abstract}` aggiungono semantica.

Le regole di visibilità dei linguaggi non coincidono sempre con UML: in un design software conviene adottare convenzioni compatibili con il linguaggio target e spesso limitarsi a + e -.

7.1.3 Molteplicità

- 1: esattamente uno, valore predefinito;
- 0..1: opzionale;
- * o 0..*: qualsiasi quantità, anche zero;
- 1..*: almeno uno;
- n..m: intervallo limitato.

Un attributo multivalore è concettualmente un insieme; **{ordered}** specifica che l'ordine è significativo. Per valori con comportamento o semantica propri è spesso preferibile introdurre un datatype, per esempio **CifraTelefono**, invece di usare una stringa generica.

7.1.4 Operazioni

La sintassi è:

```
visibilità nome(direzione parametro: tipo = default): tipoRitornato {proprietà}
```

La direzione è **in**, **out** o **inout**; **in** è predefinita. Le operazioni **query** osservano senza side effect, mentre i modificatori cambiano lo stato. Getter e setter ovvi possono essere omessi.

Un'operazione è la dichiarazione astratta del servizio; il **metodo** è il corpo che la implementa. Una stessa operazione astratta può avere metodi differenti nelle sottoclassi.

7.1.5 Datatype, costruttori e membri statici

Un oggetto possiede identità: due persone con gli stessi dati possono essere entità diverse. Un datatype rappresenta invece valori: due istanze di **Euro** con valore 10 sono equivalenti. In UML si usa lo stereotipo «datatype».

Attributi e operazioni statici appartengono alla classe e sono tradizionalmente sottolineati. I costruttori possono essere indicati come **create**, **make** o col nome della classe, secondo la convenzione adottata.

7.2 Associazioni e molteplicità

Un'associazione rappresenta una relazione fisica o concettuale tra classi. Può avere nome, ruoli agli estremi, molteplicità e verso di navigazione.

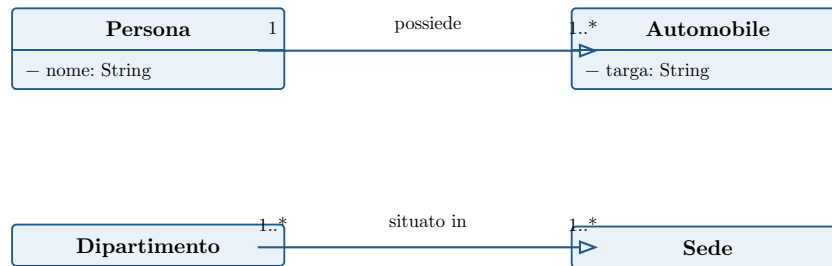


Figura 40: Associazioni orientate con nomi e molteplicità.

La freccia di navigabilità indica che, noto un oggetto all'origine, è possibile reperire quelli alla destinazione. Non coincide necessariamente con la direzione in cui si legge il nome dell'associazione.

Attenzione alle molteplicità

La molteplicità scritta accanto a una classe indica quanti oggetti di quella classe possono essere collegati a una singola istanza dell'altra estremità. Non va letta “dal proprio lato”.

7.2.1 Attributo o associazione?

La stessa informazione può talvolta essere espressa in entrambi i modi. Convenzionalmente si usano attributi per primitivi e datatype (`String`, `Date`, `boolean`) e associazioni quando il tipo è una classe con identità propria.

7.2.2 Associazioni riflessive

Una classe può essere associata a se stessa. Per esempio una cartella contiene altre cartelle o una persona svolge il ruolo di genitore rispetto ad altre persone. I nomi di ruolo sono essenziali per distinguere le estremità.

7.2.3 Associazioni bidirezionali

Una relazione navigabile in entrambe le direzioni richiede che entrambe le estremità siano mantenute coerenti. In una relazione **Man-Woman**, impostare **husband** deve aggiornare anche **wife**; un solo attributo non basta.

Costo della libertà del modello

Un design astratto e bidirezionale può sembrare semplice nel diagramma, ma trasferisce complessità alla codifica: sincronizzazione, collezioni inverse e vincoli devono essere implementati esplicitamente.

7.3 Object diagram e classi associative

7.3.1 Object diagram

Il Class Diagram descrive tutte le configurazioni valide; l'Object Diagram mostra una configurazione concreta in un istante. Un oggetto è scritto **nome : Classe**, tradizionalmente sottolineato, e un link è un'istanza di un'associazione.

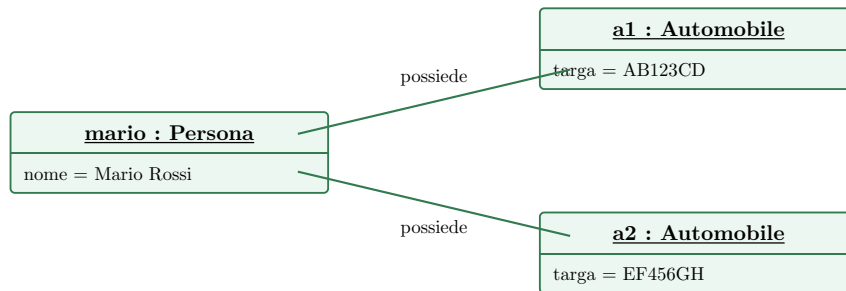


Figura 41: Snapshot di oggetti e link coerente col precedente Class Diagram.

Gli Object Diagram sono utili per spiegare class diagram complessi, verificare le molteplicità con esempi e discutere casi limite.

7.3.2 Classe associativa

Quando data, voto, quantità o altre proprietà appartengono alla relazione e non alle classi coinvolte, si usa una **association class**. In implementazione è spesso più chiaro trasformarla in una normale classe collegata alle due estremità.

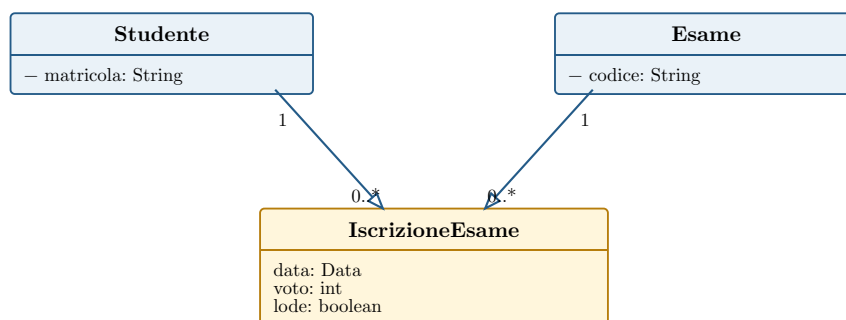


Figura 42: Le proprietà dell'associazione sono modellate come classe implementabile.

7.4 Relazioni tutto-parte e generalizzazione

7.4.1 Aggregazione e composizione

L'aggregazione rappresenta una relazione tutto-parte debole. La composizione è la forma forte: la parte non esiste senza il tutto e può appartenere a un solo composto alla volta.

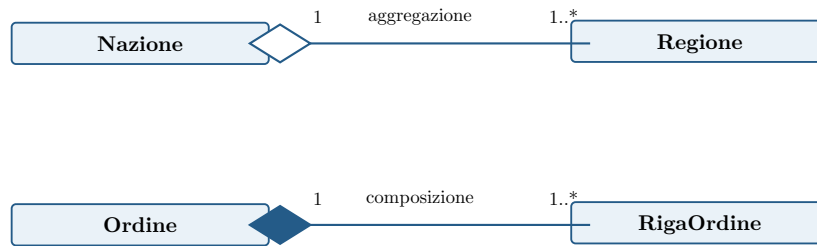


Figura 43: Diamante vuoto per aggregazione e pieno per composizione.

La distinzione dell'aggregazione è spesso ambigua e a livello software si implementa come un'associazione. Conviene usarla soprattutto quando il significato concettuale tutto-parte è davvero informativo.

7.4.2 Generalizzazione ed ereditarietà

La generalizzazione esprime una relazione concettuale “è un”: ogni istanza della sottoclasse è anche istanza della superclasse. L'ereditarietà è il meccanismo implementativo con cui la sottoclasse incorpora e specializza struttura e comportamento.

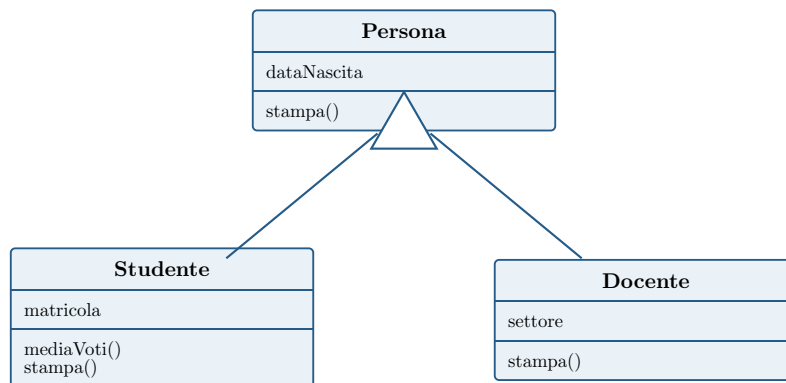


Figura 44: Le sottoclassi ereditano feature e possono aggiungerle o ridefinirle.

Un oggetto **Studente** può essere trattato come **Persona**; la sottoclasse aggiunge **matricola** e **mediaVoti()** e può ridefinire **stampa()** mediante overriding. Operazione e metodo non coincidono: l'operazione è il contratto invocabile, il metodo una sua implementazione concreta.

7.5 Dipendenze e traduzione nel codice

Una classe Cliente dipende da un Fornitore quando una modifica all'interfaccia del fornitore può richiedere una modifica al cliente. La dipendenza è rappresentata da una freccia tratteggiata.

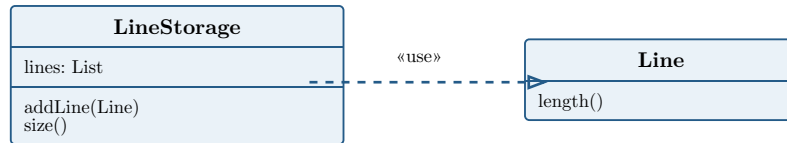


Figura 45: `LineStorage` dipende da `Line` perché la usa come tipo e parametro.

Cause comuni sono invocare operazioni, creare istanze o usare il fornitore come tipo di attributo, variabile locale, parametro o valore restituito. Nel diagramma vanno mostrate soltanto le dipendenze significative: troppe frecce nascondono la struttura che si voleva comunicare.

7.5.1 Dalle molteplicità alle strutture dati

1	Riferimento o attributo singolo.
0..1	Riferimento opzionale.
0..* / 1..*	Collezione tipizzata, per esempio <code>Set<T></code> ; il minimo deve essere validato.
{ordered} 0..*	Lista ordinata, per esempio <code>ArrayList<T></code> o <code>LinkedList<T></code> .

Non esiste una traduzione unica. Le associazioni bidirezionali richiedono due strutture sincronizzate; aggregazione e composizione richiedono politiche di ciclo di vita; vincoli e invarianti devono essere implementati o verificati.

Gestire le dipendenze

Minimizzare dipendenze e cicli, specialmente tra package. Diagrammi troppo completi sono illeggibili; strumenti automatici possono aiutare a individuare dipendenze effettive nel codice.

7.6 Riepilogo

Classe	Nome, attributi e operazioni; i dettagli possono essere soppressi.
Associazione	Relazione con ruoli, navigabilità e molteplicità.
Object Diagram	Esempio concreto di oggetti e link.
Aggregazione	Relazione tutto-parte debole.
Composizione	Tutto-parte forte con ciclo di vita condiviso.
Generalizzazione	Relazione “è un” e base per l’ereditarietà.
Dipendenza	Il cliente può essere influenzato da modifiche al fornitore.

Regola finale

Un buon Class Diagram non mostra tutto: seleziona classi, feature e relazioni necessarie alla domanda corrente, dichiara la prospettiva e usa esempi di oggetti per validare le decisioni più complesse.

8 Interfacce e vincoli UML

Il termine **interfaccia** indica sia l'insieme delle operazioni pubblicamente visibili di una classe, sia un classificatore UML simile a una classe ma privo di implementazione. Un'interfaccia dichiara un contratto; una o più classi possono fornirne realizzazioni differenti.

Idea fondamentale

Chi usa un'interfaccia dipende dal contratto, non dalla classe concreta che in quel momento lo implementa. Questo riduce l'accoppiamento e rende le implementazioni sostituibili.

8.1 Interfacce fornite e richieste

8.1.1 Realizzazione di un'interfaccia

Una classe **fornisce** un'interfaccia quando implementa tutte le operazioni che essa dichiara. UML rappresenta la realizzazione con una linea tratteggiata e un triangolo vuoto rivolto verso l'interfaccia. Lo stereotipo «interface» distingue l'interfaccia da una classe ordinaria.

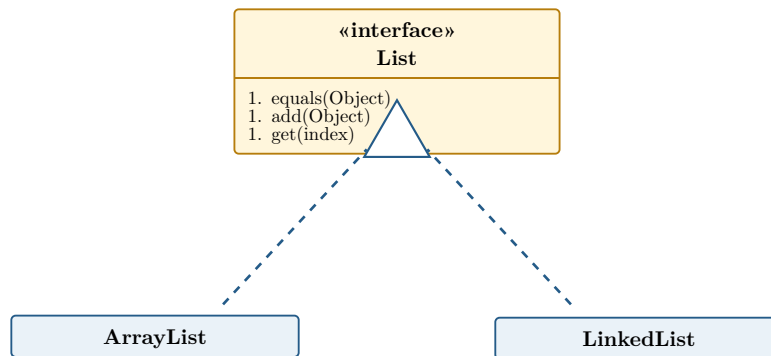


Figura 46: ArrayList e LinkedList realizzano la stessa interfaccia List.

In Java questa relazione corrisponde a `implements List`. Una classe può realizzare più interfacce; un'interfaccia può inoltre specializzarne altre.

8.1.2 Interfaccia richiesta

Una classe **richiede** un'interfaccia quando ha bisogno di un oggetto che la implementi per svolgere il proprio compito. Non richiede necessariamente una particolare classe concreta: qualunque realizzazione compatibile può soddisfare la dipendenza.

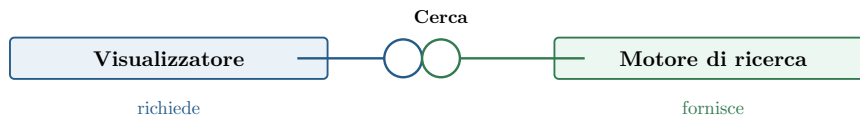


Figura 47: Il visualizzatore richiede il servizio Cerca, fornito dal motore di ricerca.

Il cerchio, detto **lollipop**, indica un'interfaccia fornita; il semicerchio o **socket** indica un'interfaccia richiesta. Quando socket e lollipop sono collegati, il componente cliente usa il servizio del fornitore.

8.2 Due notazioni equivalenti

La stessa relazione può essere espressa con la notazione estesa — rettangolo «interface» e realizzazione tratteggiata — oppure con il lollipop. La prima mostra comodamente le operazioni; la seconda è più compatta nei diagrammi architetturali.

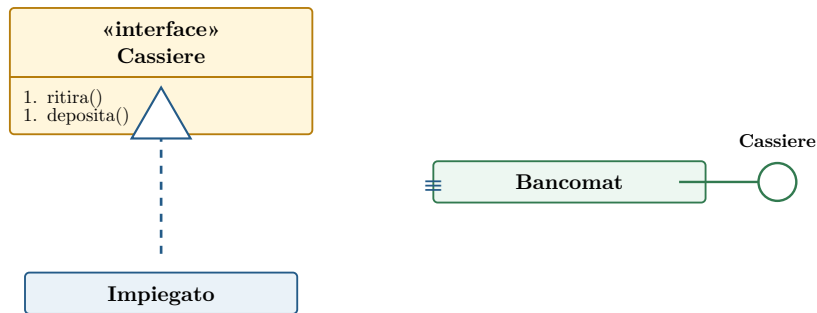


Figura 48: Notazione estesa e lollipop esprimono una realizzazione d'interfaccia.

Come scegliere la notazione

Usare il rettangolo quando interessano operazioni, generalizzazioni o dettagli del contratto; usare lollipop e socket quando interessa soprattutto mostrare come componenti e servizi sono collegati.

8.3 Perché usare le interfacce

Senza interfaccia, un cliente dipende direttamente da una classe concreta. Introducendo un contratto intermedio, sia il cliente sia le implementazioni dipendono da un'astrazione stabile.

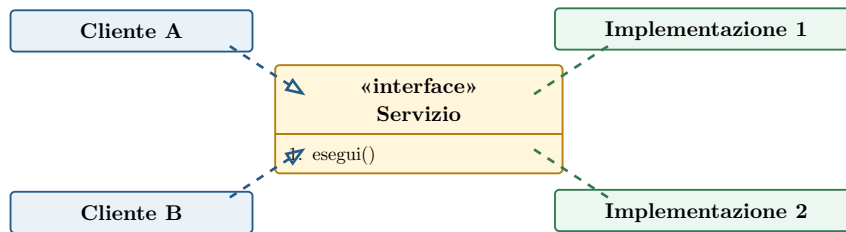


Figura 49: Clienti e implementazioni sono disaccoppiati da un contratto comune.

I principali vantaggi sono:

- **sostituibilità**: una realizzazione può essere rimpiazzata senza cambiare il cliente;
- **testabilità**: un test può fornire un'implementazione fittizia (**mock** o **stub**);
- **sviluppo parallelo**: team diversi lavorano contro un contratto concordato;
- **riduzione delle dipendenze**: il cliente conosce solo le operazioni necessarie;
- **polimorfismo**: oggetti di classi diverse sono usati uniformemente.

Interfacce troppo grandi

Un'interfaccia che raccoglie servizi non correlati costringe i clienti a dipendere da operazioni inutili. È preferibile definire contratti piccoli e coesi, orientati alle esigenze dei client.

8.4 Vincoli nei Class Diagram

Un Class Diagram non descrive soltanto elementi strutturali: associazioni, attributi, molteplicità e generalizzazioni impongono già vincoli sulle configurazioni ammesse. Per esempio, la molteplicità 1 accanto a **Persona** stabilisce che ogni automobile abbia esattamente un proprietario.

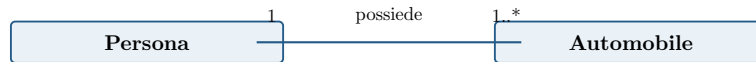


Figura 50: La molteplicità esprime il vincolo: ogni automobile appartiene a una sola persona.

I costrutti grafici non sono però sufficienti per ogni regola di business. UML consente di aggiungere vincoli mediante linguaggio naturale, codice o **OCL (Object Constraint Language)**.

8.4.1 Object Constraint Language

OCL è il linguaggio formale di specifica associato a UML, basato sulla logica del primo ordine. Può esprimere invarianti, precondizioni, postcondizioni e interrogazioni senza descrivere come implementarle.

Esempio di invariante OCL

```
context Automobile inv UnSoloProprietario: self.proprietario->size() = 1
```

Vincolo dichiarativo

OCL specifica ciò che deve essere vero, non l'algoritmo con cui ottenerlo. Il modello resta quindi indipendente dall'implementazione, ma può essere verificato con strumenti automatici.

Riepilogo

Interfaccia	Contratto pubblico privo di implementazione concreta.
Realizzazione	Una classe fornisce le operazioni dichiarate dall'interfaccia.
Interfaccia richiesta	Il cliente necessita di una realizzazione del contratto.
Lollipop/socket	Notazione compatta per servizi forniti e richiesti.
Vincolo	Regola che limita le configurazioni ammesse dal modello.
OCL	Linguaggio formale e dichiarativo per vincoli UML.

9 State Machine Diagram

Uno **State Machine Diagram** descrive il comportamento di un'entità come variazione del suo stato interno in risposta a eventi. L'entità può essere un sistema software o hardware, una classe, un singolo oggetto o un'entità del mondo reale.

La domanda del diagramma

Dato lo stato corrente e un evento ricevuto, quale transizione può scattare, quali azioni vengono eseguite e quale sarà il nuovo stato?

9.1 Stato e comportamento

9.1.1 Modellazione statica e dinamica

Class e Object Diagram descrivono entità e relazioni, cioè la struttura statica. I diagrammi di interazione mostrano più oggetti che collaborano per un obiettivo; una macchina a stati segue invece un'entità durante il suo ciclo di vita.

Uno **stato** è una situazione durante la quale valgono determinate condizioni e possono essere eseguite attività. Non coincide con la combinazione completa dei valori dei campi: è un'astrazione che raggruppa configurazioni nelle quali l'entità reagisce nello stesso modo agli eventi.

Esempio del distributore

Premere “eroga” produce caffè se il credito è sufficiente, ma mostra un messaggio se è insufficiente. Lo stesso evento genera comportamenti diversi perché cambia lo stato interno.

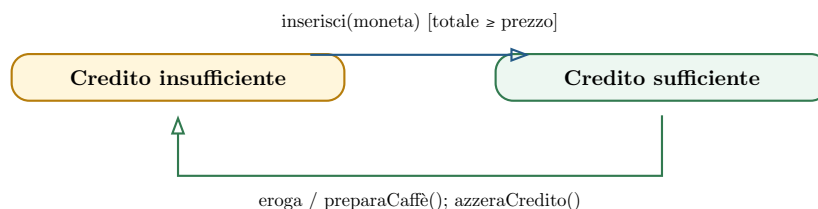


Figura 51: Il comportamento dipende sia dall'evento sia dallo stato corrente.

9.2 Notazione e semantica

9.2.1 Stati, transizioni e pseudo-stati

Graficamente una macchina a stati è un grafo: i nodi arrotondati sono stati e gli archi orientati sono transizioni. Il disco nero è lo pseudo-stato iniziale, unico nel diagramma o nello stato composito; il disco nero bordato è uno stato finale e può comparire più volte.

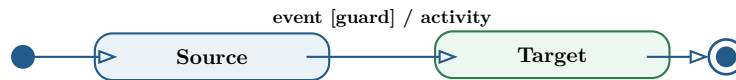


Figura 52: Sintassi fondamentale di una transizione UML.

La forma completa dell'etichetta è:

evento [guardia] / attività

- l'**evento** è il trigger che propone il passaggio;
- la **guardia** è una condizione booleana che deve risultare vera;
- l'**attività** è l'azione atomica eseguita durante la transizione.

Tutti e tre gli elementi sono opzionali. Senza attività si cambia soltanto stato; senza guardia l'evento abilita sempre la transizione; senza evento la transizione è di completamento e scatta immediatamente quando lo stato sorgente termina.

9.2.2 Eventi

Un evento può essere:

- la ricezione di un segnale o di una chiamata a operazione;
- un **change event**, scritto `when(condizione)`, quando la condizione passa da falsa a vera;
- un **time event**, come `after(2 min)` oppure `at(24:00)`.

Le azioni possono usare attributi, operazioni, link e parametri conosciuti dall'oggetto modellato. Riferirsi direttamente a elementi che l'oggetto non conosce rende il modello non implementabile.

9.2.3 Run-to-completion

Gli eventi ricevuti vengono accodati e processati uno alla volta. La semantica è **run-to-completion**: un nuovo evento viene estratto solo quando l'elaborazione del precedente è terminata. Se più transizioni sono contemporaneamente abilitate, ne viene scelta una sola; un modello con guardie sovrapposte può quindi essere non deterministico.

Guardie mutuamente esclusive

Da uno stesso stato, transizioni con lo stesso evento dovrebbero avere guardie disgiunte e possibilmente complete. Se due guardie sono vere, la scelta non è deterministica; se nessuna è vera, l'evento viene ignorato.

9.3 Esempio: ciclo di vita di una copia di libro

La macchina seguente è associata a una singola `CopiaLibro`. Mostra una prospettiva software: eventi e azioni sono operazioni e modifiche dello stato dell'oggetto.

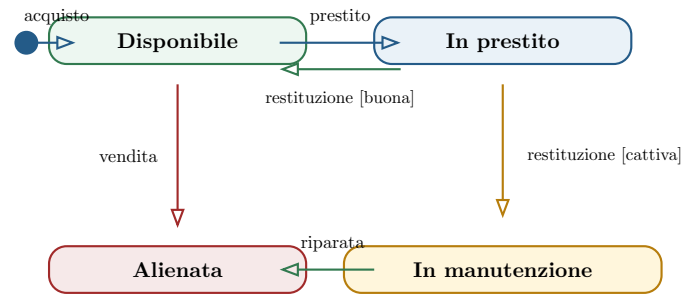


Figura 53: Macchina a stati semplificata di una copia di libro.

La restituzione produce due transizioni alternative: una copia in buone condizioni torna disponibile; una copia danneggiata entra in manutenzione. L'azione associata alla vendita può aggiornare il libro collegato se quella alienata era l'ultima copia disponibile.

9.4 Attività interne

9.4.1 Entry, exit, do e reazioni interne

Il secondo comparto di uno stato può contenere attività che non producono un cambiamento di stato:

- **entry** / azione: eseguita ogni volta che si entra;
- **exit** / azione: eseguita ogni volta che si esce;
- **do** / attività: eseguita mentre lo stato è attivo, può durare ed essere interrotta;
- **evento** / azione: reazione interna a un evento.

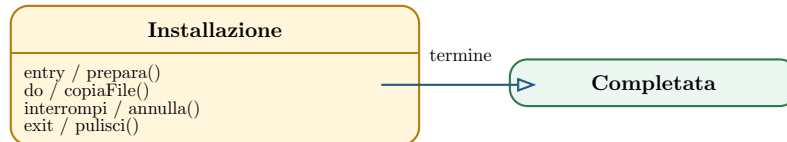


Figura 54: Uno stato può dichiarare attività di ingresso, permanenza, uscita e reazioni interne.

Una **self-transition** esce dallo stato e vi rientra, quindi esegue **exit** ed **entry**. Una reazione interna non abbandona lo stato e non lo attiva: le due notazioni non sono equivalenti.

9.5 Stati composti e concorrenza

9.5.1 Stati composti

Uno stato composto contiene una macchina a stati interna. Permette di nascondere dettagli e suddividere un comportamento complesso. Una transizione disegnata dal bordo del super-stato è disponibile da tutti i suoi sotto-stati; sono possibili anche transizioni da uno stato interno verso l'esterno.

Gli **entry point** e gli **exit point** consentono di entrare o uscire dal composto in punti nominati diversi. Una giunzione (**junction**) compatta ramificazioni e ricongiungimenti condizionali.

9.5.2 Regioni ortogonali

Un composto diviso in regioni ortogonali modella comportamenti concorrenti. L'entità si trova contemporaneamente in uno stato di ogni regione; l'uscita dal composto avviene normalmente quando tutte le regioni hanno raggiunto il proprio stato finale.

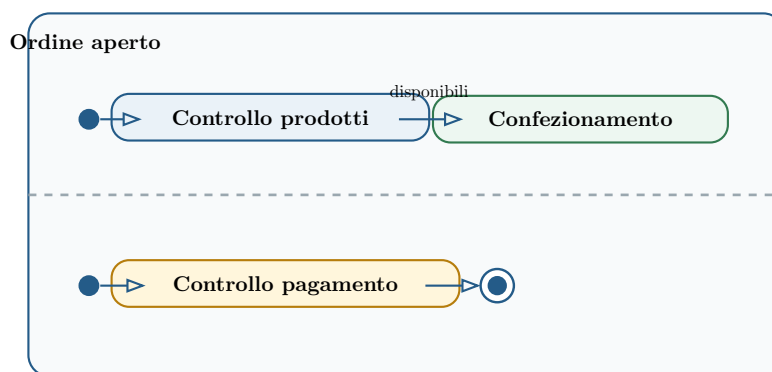


Figura 55: Due regioni ortogonali controllano prodotti e pagamento in concorrenza.

Concorrenza, non necessariamente parallelismo

Le regioni esprimono evoluzioni logicamente concorrenti. L'implementazione può usare thread diversi oppure interleaving su un singolo thread, purché rispetti la semantica del modello.

9.6 Quando usare una State Machine

Le macchine a stati sono utili quando un'entità possiede una logica interna significativa e la validità delle operazioni dipende dalla sua storia. Esempi tipici sono controllori automatici, distributori, protocolli, workflow documentali, dispositivi medicali e interfacce grafiche.

Sono meno utili per classi prevalentemente passive, servizi stateless o oggetti il cui comportamento non cambia sensibilmente nel tempo.

9.6.1 Esempio: controllore di un forno

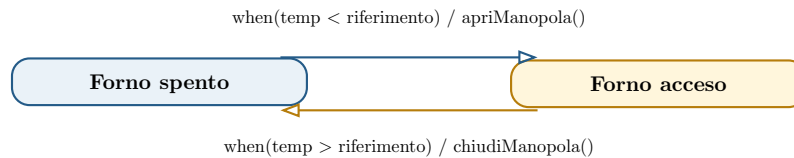


Figura 56: Il controllore reagisce agli eventi di cambiamento della temperatura.

9.6.2 Esempio: workflow documentale

Un documento può attraversare gli stati *In attesa di upload*, *In attesa di assegnazione*, *In revisione* e *In attesa di approvazione*. L'approvazione termina il flusso; il rifiuto lo riporta in revisione. Le azioni — upload, assegnazione, revisione e approvazione — appartengono alle transizioni.

Prima di costruire il diagramma conviene:

1. scegliere l'entità e la prospettiva;
2. individuare gli stati in cui cambia la risposta agli eventi;
3. elencare gli eventi significativi;
4. aggiungere transizioni e guardie;
5. completare con azioni, stati composti e concorrenza solo quando necessari;
6. verificare casi limite, eventi inattesi e raggiungibilità degli stati.

9.7 Implementazione

Nei linguaggi che non supportano direttamente le macchine a stati si usano principalmente tre strategie.

9.7.1 Switch sullo stato corrente

Un attributo memorizza lo stato; ogni metodo-evento esegue uno `switch` e seleziona la transizione valida. È diretto e adatto a modelli piccoli, ma diventa laborioso quando aumentano stati, eventi e modifiche.

```
void restituzione() {  
    switch (statoCorrente) {  
        case IN_PRESTITO:  
            if (condizione == BUONA) statoCorrente = DISPONIBILE;  
            else statoCorrente = IN_MANUTENZIONE;  
            break;  
        default:  
            throw new TransizioneNonValida();  
    }  
}
```

9.7.2 Design Pattern State

Ogni stato è rappresentato da una classe che implementa una stessa interfaccia. Il contesto delega allo stato corrente e il polimorfismo seleziona il comportamento. La soluzione separa le responsabilità e rende più semplici le estensioni, ma introduce più oggetti e classi.

9.7.3 Tabelle di stato e generazione

Una tabella elenca sorgente, destinazione, evento, guardia e azione. È una rappresentazione regolare, verificabile e adatta alla generazione automatica del codice con strumenti model-driven.

Sorgente	Destinazione	Evento	Guardia	Azione
Disponibile	In prestito	prestito	—	registra
In prestito	Disponibile	restituzione	buona	ricollocata
In prestito	Manutenzione	restituzione	cattiva	segnala

Strumenti come SMC trasformano la tabella in una descrizione formale e generano la logica della macchina, spesso secondo il pattern State. Lo sviluppatore implementa poi le azioni applicative ed espone gli eventi come metodi.

Riepilogo

Stato	Astrazione delle situazioni con uguale comportamento agli eventi.
Transizione	evento [guardia] / attività; ogni parte è opzionale.
Semantica	Eventi accodati ed elaborati uno alla volta, run-to-completion.
Attività interne	entry, exit, do e reazioni senza cambio di stato.
Stato composito	Raggruppa sotto-stati e può contenere regioni concorrenti.
Implementazione	Switch, pattern State oppure tabella con generazione automatica.